S.no	**Topic**	Subtopic (Project work included)
2	**Javascript**	**1) JavaScript Introduction** **2) Understanding of JavaScript - 1 (Functions,Events, <script> tag)** **3) Understanding of JavaScript - 2 (How to display JavaScript Output?,)** **4) Understanding of JavaScript - 3 (Javascript {Programs, Statements, Keywords})** **5) Understanding of JavaScript - 4 (JavaScript Syntax,Case Sensitive,,Identifiers)** **6) JavaScript Comment** **7) JavaScript Variables** **8) Javascript Let** **9) JavaScript const** **10) JavaScript Operators** **11) JavaScript Arithmetic** **12) JavaScript Assignment** **13) JavaScript Data Types** **14) JavaScript Functions** **15) JavaScript Objects** **16) JavaScript Events** **17) JavaScript Strings** **18) JavaScript String method - 1** **19) JavaScript String method - 2** **20) JavaScript String method - 3** **21) JavaScript String Search** **22) JavaScript String Templates** **23) JavaScript Numbers** **24) JavaScript BigInt**

		25) JavaScript Number methods **26) JavaScript Number properties** **27) Javascript Arrays** **28) JavaScript Dates** **29) JavaScript Math** **30) JavaScript Random** **31) Javascript Booleans** **32) Javascript Comparisons** **33) Javascript Hoisting** **34) Javascript Arrow Function** **35) Javascript RegExp** **36) JavaScript Scope** **37) Javascript this keyword** **38) Javascript call, apply and bind** **39) Javascript Sets** **40) Javascript Maps** **40) Object Definition & Properties** **42) Object methods** **43) Object Display** **44) Object Accessors** **45) Object Constructors** **46) Object Prototypes** **47) Object Iterables** **48) Object Reference** **49) Function Definitions** **50) Function Parameters**

		51) Function Invocation **52) Function Closures** **53) Javascript Callbacks** **54) Javascript Asynchronous** **55) Javascript Promises** **56) Javascript Async/Await** **57) DOM methods** **58) DOM elements** **59) DOM events** **60) DOM event listener** **61) DOM Nodes**

1. JavaScript Introduction

1.1 What is JavaScript?

JavaScript is a **lightweight, interpreted programming language** used to create **dynamic** and **interactive** web content. It runs directly in the browser, allowing developers to build user-driven web interfaces with real-time behavior.

It is one of the **core technologies of the web**, along with HTML and CSS.

1.2 Why Use JavaScript?

- To **respond to user actions** (e.g. clicking a button)
- To **validate forms** before they are submitted
- To **update content** without reloading the page
- To create **animations** or **dynamic effects**

1.3 Use Case Scenario:

You want a web page button to show a welcome alert when clicked. HTML and CSS can create the structure and style the button — but JavaScript brings it to life with interactivity.

1.4 How JavaScript Works with HTML<Script>

JavaScript is embedded or linked to an HTML document using the **<script>** tag.

- **INLINE**: Written directly in the HTML file

```
<script>
  alert("Hello from script tag!");
</script>
```

- **EXTERNAL**: Linked via a .js file

```
<script src="script.js"></script>
```

Practical Example: A Simple JavaScript Alert

```
<!DOCTYPE html><html>
<head><title>JavaScript Introduction</title></head>
<body>
<button onclick="showMessage()">Click Me</button>
<script>
  function showMessage() {
    alert("Hello! You clicked the button.");
  }
</script>
</body></html>
```

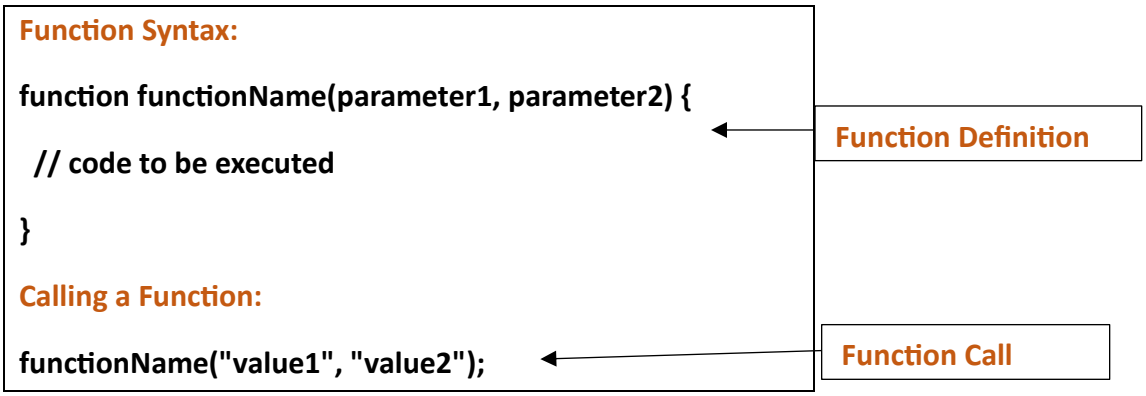
Explanation:

- The <button> element triggers an event when clicked.
- The onclick attribute calls the showMessage() function.
- The function uses the built-in alert() method to display a message.

2. Understanding of JavaScript – 1

2.1 What is a Function in JavaScript?

A **function** is a reusable block of code designed to perform a specific task. It is executed when **called (invoked)**.



2.2 What are JavaScript Events?

Events are actions that occur in the browser — such as clicking, hovering, loading, or typing — that JavaScript can respond to.

Event	Triggered When...
onclick	An element is clicked
onmouseover	The mouse moves over an element
onmouseout	The mouse leaves an element
onload	The page has finished loading
onchange	A form element's value changes

Practical Example:

```
<!DOCTYPE html><html><head>
  <title>Functions & Events</title>
</head>
<body>
<h2>Function and Event Demo</h2>
<p id="message">Original Text</p>
<button onclick="updateText()">Click Me</button>
<script>
  function updateText() {
    document.getElementById("message").innerHTML = "You
clicked the button!";
  }
</script>
</body></html>
```

Explanation: The function **updateText()** changes the content of the <p> tag.

- The button uses the onclick event to call the function.
- JavaScript is written inside a <script> tag in the HTML file.

2.3 Understanding of JavaScript - 2

How to Display JavaScript Output?

JavaScript provides several ways to display output, depending on where and how you want the result shown. The most common output methods are:

2.3.1 innerHTML

Used to write content directly inside an **HTML element**.

```
document.getElementById("demo").innerHTML = "Hello, World!";
```

Best for: **Updating the page content dynamically**

2.3.2 document.write()

Writes directly into the **HTML document**.

```
document.write("This is a document write.");
```

Not recommended for use after the page has loaded — it can overwrite the entire document.

2.3.3 alert()

Displays an **alert box** (popup window) with a message.

```
alert("Welcome to JavaScript!");
```

Best for: **Simple messages or warnings**

2.4.4 console.log()

Logs data to the **browser console** (useful for developers).

```
console.log("This message appears in the console.");
```

Best for: **Debugging during development**

Use Case Scenario: Update content on the page,Show an alert to the user, Log something to the console,Optionally write directly to the document (rarely used)

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Output</title>
</head>
<body>
<h2>Output Methods</h2>
<p id="output"></p>
<button onclick="showOutput()">Show Output</button>
<script>
  function showOutput() {
    // 1. Display in HTML element
    document.getElementById("output").innerHTML = "Displayed
using innerHTML";

    // 2. Display as popup
    alert("This is an alert!");

    // 3. Display in console
    console.log("This message is logged in the console");

    // 4. Write directly to the document (only during load)
    document.write("<p>This is document.write()</p>");
  }
</script>
</body>
</html>
```

Explanation: innerHTML updates the <p> tag.

- alert() displays a popup.
- console.log() logs the message to browser tools (F12 → Console).
- document.write() writes directly to the document but is destructive if used after the page loads.

4. Understanding of JavaScript - 3

4.1 What is a JavaScript Program?

A **JavaScript program** is a set of one or more **statements** that tell the browser what to do. These statements are executed **sequentially** from top to bottom unless control structures (like loops or conditions) change the flow.

A complete JavaScript file or script in your HTML page is considered a program.

4.2 What is a Statement in JavaScript?

A **statement** is a single instruction that performs an action.

Every JavaScript statement **ends with a semicolon (;)** (although semicolons are optional in many cases, using them is a best practice).

Example:

```
let x = 10;     // This is a statement

let y = 20;     // Another statement

let sum = x + y; // Third statement
```

Statements can:

- Declare variables & Assign values
- Perform operations
- Define functions, Control logic (if, loops, etc.)

4.3 JavaScript Code Blocks

Statements can be grouped using **curly braces {}** to form code blocks — useful inside if, for, and function definitions.

```
function greet() {

  let name = "Aman";

  alert("Hello " + name);}
```

4.4 JavaScript Keywords

JavaScript has a set of **reserved words** (keywords) that are part of the language syntax. You **cannot use them as variable or function names**.

These keywords are **case-sensitive**, and misusing them can break your script.

Keyword	Description
var	Declares a variable
let	Declares a block-scoped variable
const	Declares a constant (cannot change)
function	Declares a function
return	Returns a value from a function
if / else	Conditional branching
for, while	Looping constructs
break	Exits the current loop early
continue	Skips the current loop iteration
true, false	Boolean values
null / undefined	Special variable values
typeof	Returns the data type of a value

Use Case Scenario: You want to write a small program that:

- Declares two variables
- Adds their values using a statement
- Uses **keywords like let and document.getElementById** to show the result

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JS Statements & Keywords</title>
</head>
<body>
<h2>JavaScript Program with Statements</h2>
<p id="result"></p>
<script>
  // JavaScript Program starts here
  // Declare variables using keywords
  let a = 15;       // Statement 1
  let b = 25;       // Statement 2
  // Perform an operation
  let total = a + b; // Statement 3
  // Display the result
  document.getElementById("result").innerHTML =
    "The sum of a and b is: " + total; // Statement 4
</script>
</body>
</html>
```

Explanation:

1. **let a = 15;** — declares and initializes variable a
2. **let b = 25;** — declares and initializes variable b
3. **let total = a + b;** — calculates the sum
4. **document.getElementById(...)** — shows the output in the HTML

5. Understanding of JavaScript - 4

5.1 JavaScript Syntax

JavaScript syntax is the set of rules that defines **how JavaScript programs are constructed**.

Basic syntax rules:

- JavaScript is **case-sensitive**.
- Statements should **end with semicolons (;)**.
- **Blocks of code** are enclosed within **curly braces {}**.
- **Functions**, **variables**, and **object keys** all follow specific naming rules.

Example:

```
let name = "Aman";


function greet() {
  alert("Hi " + name);
}
```

5.2 JavaScript is Case-Sensitive

This means user, User, and USER are **three different variables**.

Example:

```
let name = "Aman";
let Name = "Ravi";
alert(name); // Aman
alert(Name); // Ravi
```

Use Case Scenario:

- Declare some variables using valid identifiers
- Show the difference between case-sensitive variable names
- Add comments to describe what the code does

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JS Syntax & Variables</title>
</head>
<body>
<h2>JavaScript Syntax & Case Sensitivity</h2>
<p id="caseOutput"></p>
<script>
// Declare variables

  let student = "Anjali";
  let Student = "Karan";

  // Show how case matters

  document.getElementById("caseOutput").innerHTML =
    "student = " + student + "<br>" + "Student = " + Student;
</script>
</body>
</html>
```

Summary:

- JavaScript syntax must follow strict structure rules
- Comments improve code readability and help with debugging
- Variable names (identifiers) must follow naming conventions
- JavaScript is **case-sensitive**, so myVar ≠ MyVar

5.3 JavaScript Identifiers

Identifiers are the **names** used for: Variables, Functions, Arrays, Objects

Naming Rules:

- Must begin with a **letter**, _, or $

- Cannot start with a **digit**
- Cannot use **reserved keywords** (like let, for, if, etc.)
- Are **case-sensitive**

Valid Identifiers:

```
let userName;
let _privateVar;
let $dollarVar;
```

Invalid Identifiers:

```
let 2user;   // starts with number
let let;     // reserved word
```

6. JavaScript Comments

Comments are **ignored by the browser** and used to explain code or disable parts of code for debugging. **Comments** are lines in JavaScript code that are not executed by the browser.

- Explain the code
- Make it easier to read and maintain
- Temporarily disable code during debugging

Types of Comments:

Type	Syntax	Usage
Single-line	// This is a comment	Comment out one line of code or note
Multi-line	/* This is a block comment */	Comment out multiple lines

Use Case Scenario: You're working in a team and want to leave notes or explanations for others (or yourself later).

Example:

```
// This is a single-line comment
let x = 10;
/*
 This is a multi-line comment
 describing a section of code
*/
let y = 20;
```

7. JavaScript Variables

Variables are containers for storing data. You can declare them using var, let, or const.

Quick Rules for Naming Variables in JavaScript:

- Must begin with a **letter**, _, or $
- Can contain letters, numbers, _, and $
- Cannot begin with a **digit (0–9)**
- Cannot contain **spaces**, **punctuation**, or **special symbols** (like @, !, -)
- Cannot use **JavaScript reserved keywords** (e.g., let, var, return, function)

Variable Name	Valid / Invalid	Reason
username	Valid	Starts with a letter
_user	Valid	Starts with an underscore
$value	Valid	Starts with a dollar sign
user123	Valid	Alphanumeric characters allowed
camelCase	Valid	Recommended style for naming variables

Variable Name	Valid / Invalid	Reason
first_name	Valid	Underscore usage is allowed
2ndPlace	Invalid	Cannot begin with a digit
var	Invalid	var is a **reserved keyword**
let	Invalid	let is a **reserved keyword**
full-name	Invalid	Hyphens (-) are not allowed (interpreted as minus sign)
user name	Invalid	Spaces are not allowed in variable names
@admin	Invalid	Cannot begin with special characters (except _ and $)
function123	Invalid	function is a reserved keyword; cannot start with it directly
user!score	Invalid	! is not allowed in variable names

Example:

```
let age = 25;


const PI = 3.14;

var city = "Delhi";
```

Keyword	Scope Type	Can Reassign?	Redeclare?	Introduced In
var	Function	Yes	Yes	ES5
let	Block	Yes	No	ES6
const	Block	No	No	ES6

8. JavaScript let

The let keyword allows you to **declare a block-scoped variable** — meaning it is only accessible within the block or {} where it's defined.

Introduced in **ES6 (ECMAScript 2015)**, it's a modern alternative to var.

Feature	Description
Block-scoped	Works only within {} in which it is declared
Cannot be redeclared	In the same block
Can be reassigned	Unlike const, the value can change

Use Case Scenario: You want to create a variable inside a loop or function without it affecting other blocks.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
 <title>let Keyword</title>
</head>
<body>
<p id="letDemo"></p>
<script>
 let greeting = "Hello";
 if (true) {
  let greeting = "Hi"; // different variable (block scoped)

  console.log(greeting); // Hi
 }
 console.log(greeting); // Hello
 document.getElementById("letDemo").innerHTML = greeting;
</script>
</body></html>
```

9. JavaScript const

The const keyword declares a **read-only block-scoped variable**. Once a value is assigned, it **cannot be reassigned**.

Feature	Description
Must be initialized	Cannot declare without assigning a value
Block-scoped	Like let, limited to the block where it's declared
Cannot be reassigned	Once assigned, the value is final
Object properties can change	The object itself is constant, but its contents can mutate

Use Case Scenario:

You want to declare a variable whose value should never change — such as configuration values, fixed IDs, or mathematical constants.

Practical Example:

```
<!DOCTYPE html><html><head>
  <title>const Keyword</title>
</head>
<body>
<p id="constDemo"></p>
<script>
  const PI = 3.14159;
  // This will throw an error:
  // PI = 3.15;
  const person = {
    name: "Aman",
    age: 25
  };
  // We can still change the properties:
  person.age = 26;
  document.getElementById("constDemo").innerHTML =
    person.name + " is " + person.age + " years old.";
</script>
</body></html>
```

10. JavaScript Operators

Operators perform operations on values and variables. JavaScript supports various types of operators:

1. **Arithmetic Operator**:- Used for mathematical calculations.
2. **Assignment Operator:-** used to Assign values to variables.
3. **Comparison Operator:-** used to compare two values & return Boolean.
4. **Logical Operator:-** used to combine multiple conditions.
5. **String Operator**:- used to concat the strings.
6. **Type Operator:-** used to check the data type of the variable.
7. **Bitwise Operator:-** Operate on 32-bit binary numbers.
8. **Ternary Operator:-** A shortcut for if-else.

10.1 JavaScript Arithmetic

JavaScript arithmetic operators are used to **perform mathematical operations** on numbers (both literals and variables).

Use Case Scenario:

You want to calculate the total price of products or evaluate mathematical expressions in a script.

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 2	3
%	Modulus (Remainder)	5 % 2	1
++	Increment	x++	x = x + 1
--	Decrement	x--	x = x - 1

Practical Example:

```
<!DOCTYPE html><html>
<head><title>Arithmetic Operators</title></head>
<body>
<p id="math"></p>
<script>
  let x = 10;
  let y = 3;
  let total = x + y;
  let mod = x % y;
  document.getElementById("math").innerHTML =
    "Total: " + total + "<br>" +
    "Modulus: " + mod;
</script>
</body></html>
```

10.2 Assignment Operators

Assignment operators are used to **assign values** to JavaScript variables.

Use Case Scenario:

Updating a total score, price, or any value that accumulates or changes over time.

Operator	Example	Same As
=	x = y	Assign y to x
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Practical Example:

```
<!DOCTYPE html>
<html>
<head><title>Assignment Operators</title></head>
<body>
<p id="assign"></p>
<script>
  let x = 10;
  x += 5; // now x is 15
  let y = 4;
  y *= 2; // now y is 8
  document.getElementById("assign").innerHTML =
    "x: " + x + "<br>" +
    "y: " + y;
</script>
</body>
</html>
```

10.3 Comparison Operators

Used to compare two values and return a boolean.

Operator	Description	Example	Result
==	Equal to (loose)	5 == "5"	true
===	Equal value and type	5 === "5"	false
!=	Not equal	5 != "5"	false
!==	Not equal (value & type)	5 !== "5"	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal to	5 >= 5	true
<=	Less than or equal to	5 <= 3	false

10.4 Logical Operators

Used to combine multiple conditions.

Operator	Description	Example	Result
&&	Logical AND	true && false	false
`	Logical OR	`	Logical OR
!	Logical NOT	!true	false

Use Case Scenario: You want to perform calculations and compare values using arithmetic and logical operators.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Operators</title>
</head>
<body>
<p id="operatorDemo"></p>
<script>
  let x = 10;
  let y = 5;
  let result = (x + y) * 2; // Arithmetic
  let isEqual = x == y;     // Comparison
  let isGreater = x > y;    // Comparison
  let isValid = (x > 0 && y > 0); // Logical
  document.getElementById("operatorDemo").innerHTML =
    "Result: " + result + "<br>" +
    "Equal: " + isEqual + "<br>" +
    "x > y: " + isGreater + "<br>" +
    "Both Positive: " + isValid;
</script>
</body>
</html>
```

10.5 String Operators

Used to concatenate strings.

Operator	Description	Example	Result
+	Concatenation	"Hello" + " World"	"Hello World"
+=	Add and assign	greeting += "!"	Appends to string

10.6 Type Operators

Used to determine data type or convert type.

Operator	Description	Example	Result
typeof	Returns type of a variable	typeof 123	"number"
instanceof	Tests if an object is an instance of a constructor	x instanceof Array	true/false

10.7 Bitwise Operators (Advanced Use)

Operate on 32-bit binary numbers.

Operator	Description	Binary Example	Result
&	AND	5 & 1 → 0101 & 0001 → 0001 → 1	
`	`	OR	`5
^	XOR	5 ^ 1 → 0101 ^ 0001 → 0100 → 4	
~	NOT	~5	-6
<<	Left shift	5 << 1	10
>>	Right shift	5 >> 1	2

10.8 Ternary Operator

A shortcut for if-else.

Syntax	Example	Result
condition ? val1 : val2	age > 18 ? "Adult" : "Minor"	"Adult" if age > 18

11. JavaScript Data Types

JavaScript variables can hold **different data types**, such as numbers, strings, booleans, objects, arrays, etc.

Data Type	Example	Description
Number	let age = 25;	Integer or float
String	let name = "Aman";	Text enclosed in quotes
Boolean	let isTrue = true;	Logical value (true/false)
Array	let colors = ["red", "blue"];	List-like structure
Object	let user = {name:"Ali", age:30};	Key-value pairs
Null	let temp = null;	Empty or unknown value
Undefined	let test;	Declared but not assigned
Symbol	let sym = Symbol("id");	Unique immutable identifier

Use Case Scenario:

You need to choose the correct type to store a name (string), age (number), status (boolean), or user profile (object).

Practical Example:

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Data Types</title></head>
<body>
<p id="types"></p>
<script>
  let name = "Aman";      // String
  let age = 25;           // Number
  let isStudent = true;   // Boolean
  let colors = ["red", "green", "blue"]; // Array
  let user = {name: "Aman", age: 25};    // Object
  document.getElementById("types").innerHTML =
    "Name: " + name + "<br>" +
    "Age: " + age + "<br>" +
    "Student: " + isStudent + "<br>" +
    "First Color: " + colors[0] + "<br>" +
    "User Name: " + user.name;
</script>
</body>
</html>
```

12. JavaScript Functions

A **function** is a reusable block of code that performs a specific task when invoked.

Function Syntax:

```
function functionName(parameter1, parameter2) {
  // code block
}
```

Function Invocation:

```
functionName(arg1, arg2);
```

Use Case Scenario:

You want to reuse a set of instructions (like calculating a sum or greeting users) throughout your application.

Practical Example:

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Functions</title></head>
<body>

<p id="functionDemo"></p>

<script>
  function greet(name) {
    return "Hello, " + name + "!";
  }

  document.getElementById("functionDemo").innerHTML =
greet("Aman");
</script>

</body>
</html>
```

Function Defining

Function Call

15. JavaScript Objects

JavaScript objects are **collections of key-value pairs**, used to store and group related data.**Object Syntax:**

```
let person = {
  firstName: "Aman",
  lastName: "Sharma",age: 25};
```

Accessing properties: `person.firstName // Aman`

Use Case Scenario:

You want to store details about a user, such as name, age, email, etc., in a single variable.

Practical Example:

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Objects</title></head>
<body>

<p id="objectDemo"></p>

<script>
  let user = {
    name: "Aman",
    age: 25,
    city: "Delhi"
  };

  document.getElementById("objectDemo").innerHTML =

    user.name + " is " + user.age + " years old and lives in " +
user.city;

</script>

</body>
</html>
```

16. JavaScript Events

An **event** is a signal that something has happened — such as a button click, mouse movement, or form submission — which JavaScript can respond to.

Event	Description
onclick	User clicks an element
onmouseover	Mouse hovers over an element
onmouseout	Mouse leaves an element
onchange	Form element value changes
onload	Page or image finishes loading

Event Handling Syntax:

```
<button onclick="myFunction()">Click Me</button>

<script>

 function myFunction() {

  alert("You clicked me!");

 }

</script>
```

Use Case: You want an alert to appear or a section of content to update when a user interacts with your webpage.

Practical Example:

```
<!DOCTYPE html><html>
<head><title>JavaScript Events</title></head>
<body>
<button onclick="showMessage()">Click Here</button>
<p id="eventText"></p>
<script>
 function showMessage() {
  document.getElementById("eventText").innerHTML = "Button
was clicked!";
 }
</script>
</body></html>
```

17) JavaScript Strings

A String is a sequence of characters used to represent text.

Syntax:

```
let text = "Hello World";
```

Practical:-

```
<!DOCTYPE html>
<html>
<head>

 <title>JavaScript Strings Example</title>

</head>

<body>

<h2>JavaScript String Example</h2>
<p id="demo"></p>

<script>
 let firstName = "John";
 let lastName = "Doe";
 let fullName = firstName + " " + lastName;
 document.getElementById("demo").innerHTML = fullName;
</script>

</body>
</html>
```

Output:-

John Doe

18) JavaScript String Methods - 1

18.1. length:- Returns the number of characters in a string.

Syntax:

```
string.length
```

Practical :

```
<!DOCTYPE html>
<html>
<head>
  <title>String Length Example</title>
</head>
<body>

<h2>String Length Example</h2>
<p id="lengthDemo"></p>

<script>
  let text = "Welcome!";
  let lengthOfText = text.length;

  document.getElementById("lengthDemo").innerHTML =
"Length of string: " + lengthOfText;
</script>

</body>
</html>
```

Output:-

Length of string: 8

18.2 toUpperCase():- Converts the string to **UPPERCASE** letters.

Syntax:

```
string.toUpperCase()
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>toUpperCase Example</title>
</head>
<body>

<h2>toUpperCase() Example</h2>
<p id="upperDemo"></p>

<script>
  let text = "javascript";
  let upperText = text.toUpperCase();

  document.getElementById("upperDemo").innerHTML =
upperText;
</script>

</body>
</html>
```

Output:-

JAVASCRIPT

18.3 toLowerCase() :- Converts the string to **lowercase** letters.

Syntax:

```
string.toLowerCase()
```

Practical:

```
<!DOCTYPE html><html>
<head>
  <title>toLowerCase Example</title>
</head>
<body>
<h2>toLowerCase() Example</h2>
<p id="lowerDemo"></p>
```

```
<script>
  let text = "JAVASCRIPT";
  let lowerText = text.toLowerCase();

  document.getElementById("lowerDemo").innerHTML =
lowerText;
</script>

</body>
</html>
```

Output:-

javascript

18.4 charAt(index):- Returns the character at a specified index.

Syntax:

```
string.charAt(index)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>charAt Example</title>
</head>
<body>

<h2>charAt() Example</h2>
<p id="charAtDemo"></p>

<script>
  let text = "Hello";
  let character = text.charAt(1); // 2nd character

  document.getElementById("charAtDemo").innerHTML = "Character
at index 1: " + character;
</script>

</body>
</html>
```

Output:-

Character at index 1: e

18.5 charCodeAt(index)

Returns the Unicode (ASCII) value of the character at a given index.

Syntax:

```
string.charCodeAt(index)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>charCodeAt Example</title>
</head>
<body>

<h2>charCodeAt() Example</h2>
<p id="charCodeAtDemo"></p>

<script>
  let text = "A";
  let unicodeValue = text.charCodeAt(0);

  document.getElementById("charCodeAtDemo").innerHTML =
"Unicode value of 'A': " + unicodeValue;
</script>

</body>
</html>
```

Output:-

Unicode value of 'A': 65

19) JavaScript String Methods - 2

19.1 slice(start, end)

Extracts a part of a string and returns the extracted part in a new string.

Syntax:

```
string.slice(start, end)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>slice() Example</title>
</head>
<body>

<h2>slice() Example</h2>
<p id="sliceDemo"></p>

<script>
  let text = "JavaScript Programming";
  let result = text.slice(0, 10);

  document.getElementById("sliceDemo").innerHTML = "Sliced text: "
+ result;
</script>

</body>
</html>
```

Output:-

Sliced text: JavaScript

19.2 substring(start, end)

Returns a part of the string between the start and end indexes (similar to slice(), but does not accept negative indexes).

Syntax:

```
string.substring(start, end)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>substring() Example</title>
</head>
<body>
<h2>substring() Example</h2>
<p id="substringDemo"></p>
```

```
<script>
  let text = "JavaScript Programming";
  let result = text.substring(11, 22);

  document.getElementById("substringDemo").innerHTML =
"Substring text: " + result;
</script>

</body>
</html>
```

Output:-

Substring text: Programming

19.3. substr(start, length)

Returns a part of the string, starting at the specified position and of the specified length.

Syntax:

```
string.substr(start, length)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>substr() Example</title>
</head>
<body>
<h2>substr() Example</h2>
<p id="substrDemo"></p>
<script>
  let text = "JavaScript Programming";
  let result = text.substr(0, 10);

  document.getElementById("substrDemo").innerHTML = "Subst
text: " + result;
</script>

</body>
</html>
```

Output:-

Substr text: JavaScript

19.4 concat(string2)

Joins two or more strings together.

Syntax:

```
string.concat(string2, string3, ..., stringN)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>concat() Example</title>
</head>
<body>

<h2>concat() Example</h2>
<p id="concatDemo"></p>

<script>
  let text1 = "Hello";
  let text2 = "World";
  let result = text1.concat(" ", text2);

  document.getElementById("concatDemo").innerHTML =
"Concatenated text: " + result;
</script>

</body>
</html>
```

Output:-

Concatenated text:
Hello World

19.5 trim()

Removes whitespace from both ends of a string.

Syntax:

```
string.trim()
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>trim() Example</title>
</head>
<body>

<h2>trim() Example</h2>
<p id="trimDemo"></p>

<script>
  let text = "   Hello World!   ";
  let result = text.trim();

  document.getElementById("trimDemo").innerHTML = "Trimmed
text: '" + result + "'";
</script>

</body>
</html>
```

Output:-

Trimmed text: 'Hello World!'

20) JavaScript String Methods - 3

20.1 replace(searchValue, newValue)

Replaces the first match of searchValue with newValue in a string.

Syntax:

```
string.replace(searchValue, newValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>replace() Example</title>
</head>
<body>
```

```
<h2>replace() Example</h2>
<p id="replaceDemo"></p>

<script>
  let text = "I love JavaScript!";
  let newText = text.replace("JavaScript", "Coding");

  document.getElementById("replaceDemo").innerHTML = newText;
</script>

</body>
</html>
```

Output:-

I love Coding!

20.2 replaceAll(searchValue, newValue)

Replaces **all matches** of searchValue with newValue in a string.

Syntax:

```
string.replaceAll(searchValue, newValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>replaceAll() Example</title>
</head>
<body>
<h2>replaceAll() Example</h2>
<p id="replaceAllDemo"></p>

<script>
  let text = "I love cats. Cats are amazing!";
  let newText = text.replaceAll("Cats", "Dogs");

  document.getElementById("replaceAllDemo").innerHTML =
newText;
</script>
</body>
</html>
```

Output:-

I love cats. Dogs are amazing!

20.3 split(separator)

Splits a string into an array of substrings based on the separator.

Syntax:

```
string.split(separator)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>split() Example</title>
</head>
<body>

<h2>split() Example</h2>
<p id="splitDemo"></p>

<script>
  let text = "Apple,Banana,Cherry";
  let fruits = text.split(",");

  document.getElementById("splitDemo").innerHTML = fruits.join(" | ");
</script>

</body>
</html>
```

Output:-

Apple | Banana | Cherry

20.4 repeat(count)

Returns a new string with the original string repeated count times.

Syntax:

```
string.repeat(count)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>repeat() Example</title>
</head>
<body>

<h2>repeat() Example</h2>
<p id="repeatDemo"></p>

<script>
  let text = "Hello! ";
  let repeatedText = text.repeat(3);

  document.getElementById("repeatDemo").innerHTML =
repeatedText;
</script>

</body>
</html>
```

Output:-

Hello! Hello! Hello!

21) JavaScript String Search

20.1 indexOf(searchValue)

Returns the **index** of the first occurrence of a specified text in a string. Returns -1 if not found.

Syntax:

```
string.indexOf(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>indexOf() Example</title>
</head>
<body>
```

```
<h2>indexOf() Example</h2>
<p id="indexOfDemo"></p>

<script>
  let text = "Learn JavaScript Programming";
  let position = text.indexOf("JavaScript");

  document.getElementById("indexOfDemo").innerHTML = "Position
of 'JavaScript': " + position;
</script>

</body>
</html>
```

Output:-

Position of 'JavaScript': 6

21.2 lastIndexOf(searchValue)

Returns the **index** of the last occurrence of a specified text.

Syntax:

```
string.lastIndexOf(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>lastIndexOf() Example</title>
</head>
<body>
<h2>lastIndexOf() Example</h2>
<p id="lastIndexOfDemo"></p>
<script>
  let text = "Welcome to JavaScript world. JavaScript is powerful!";
  let position = text.lastIndexOf("JavaScript");

  document.getElementById("lastIndexOfDemo").innerHTML = "Last
position of 'JavaScript': " + position;
</script>
</body>
</html>
```

Output:-

Last position of 'JavaScript': 27

21.3 search(searchValue)

Searches a string for a specified value and returns the position.

Syntax:

```
string.search(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>search() Example</title>
</head>
<body>
<h2>search() Example</h2>
<p id="searchDemo"></p>
<script>
  let text = "I love JavaScript!";
  let position = text.search("JavaScript");

  document.getElementById("searchDemo").innerHTML = "Found at
position: " + position;
</script>
</body>
</html>
```

Output:-

Found at position: 7

21.4 includes(searchValue)

Checks if a string contains a specified value. Returns true or false.

Syntax:

```
string.includes(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>includes() Example</title>
```

```
</head>
<body>

<h2>includes() Example</h2>
<p id="includesDemo"></p>

<script>
  let text = "Hello world!";
  let result = text.includes("world");

  document.getElementById("includesDemo").innerHTML = "Contains
'world': " + result;
</script>

</body>
</html>
```

Output:-

Contains 'world': true

21.5 startsWith(searchValue)

Checks if a string starts with a specified value.

Syntax:

```
string.startsWith(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>startsWith() Example</title>
</head>
<body>
<h2>startsWith() Example</h2>
<p id="startsWithDemo"></p>
<script>
  let text = "Hello JavaScript!";
  let result = text.startsWith("Hello");

  document.getElementById("startsWithDemo").innerHTML = "Starts
with 'Hello': " + result;
</script></body></html>
```

Output:-

Starts with 'Hello':

21.6 endsWith(searchValue)

Checks if a string ends with a specified value.

Syntax:

```
string.endsWith(searchValue)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>endsWith() Example</title>
</head>
<body>

<h2>endsWith() Example</h2>
<p id="endsWithDemo"></p>

<script>
  let text = "Learning JavaScript!";
  let result = text.endsWith("JavaScript!");

  document.getElementById("endsWithDemo").innerHTML = "Ends
with 'JavaScript!': " + result;
</script>

</body>
</html>
```

Output:-

Ends with 'JavaScript!': true

22) JavaScript String Templates (Template Literals)

Template Literals (` ` backticks)

Template literals allow **multiline strings** and **string interpolation** using ${}.

22.1 Basic Example of Template Literal

Practical:

```
<!DOCTYPE html><html>
<head>
  <title>Template Literals Example</title>
</head>
<body>
<h2>Template Literal - Basic Example</h2>
<p id="basicTemplateDemo"></p>

<script>
  let name = "John";
  let greeting = `Hello, ${name}! Welcome to JavaScript.`;

  document.getElementById("basicTemplateDemo").innerHTML =
greeting;
</script>
</body>
</html>
```

Output:-

Hello, John! Welcome to JavaScript.

22.2 Multiline String with Template Literal

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Template Literals - Multiline</title>
</head>
<body>

<h2>Template Literal - Multiline Example</h2>
<p id="multilineDemo"></p>

<script>
  let text = `This is line one.
This is line two.
This is line three.`;

  document.getElementById("multilineDemo").innerText = text;
</script>
</body></html>
```

Output:-

This is line one.

This is line two.

This is line three.

22.3 Expressions Inside Template Literal

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Template Literals - Expressions</title>
</head>
<body>

<h2>Template Literal - Expression Example</h2>
<p id="expressionDemo"></p>

<script>
  let price = 100;
  let tax = 0.18;
  let totalPrice = `Total price including tax: ₹${price + (price * tax)}`;

  document.getElementById("expressionDemo").innerHTML =
totalPrice;
</script>

</body>
</html>
```

Output:-

Total price including tax: ₹118

22.4 HTML Templates using Template Literal

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Template Literals - HTML Example</title>
</head>
<body>
<h2>Template Literal - HTML Code Example</h2>
<div id="htmlTemplateDemo"></div>

<script>
  let title = "JavaScript Handbook";
```

```
  let author = "OpenAI";

  let htmlContent = `
    <h3>${title}</h3>
    <p>Written by: ${author}</p>
  `;

  document.getElementById("htmlTemplateDemo").innerHTML =
htmlContent;
</script>

</body>
</html>
```

Output:-

JavaScript Handbook
Written by: OpenAI

23) JavaScript Numbers

23.1 Declaring Numbers

In JavaScript, numbers can be declared in two ways:

- **Integer numbers**
- **Floating-point numbers (decimals)**

Practical:

```
<!DOCTYPE html><html>
<head>
  <title>Declaring Numbers</title>
</head>
<body>
<h2>Declaring Numbers</h2>
<p id="numberDemo"></p>
<script>
  let integerNum = 42;
  let floatNum = 3.14;

  document.getElementById("numberDemo").innerHTML = `Integer:
${integerNum}, Float: ${floatNum}`;
</script>
</body>
</html>
```

Output:-

Integer: 42, Float: 3.14

23.2 Number() Constructor

The Number() function is used to convert a value to a number.

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number Constructor Example</title>
</head>
<body>
<h2>Number Constructor Example</h2>
<p id="numberConstructorDemo"></p>
<script>
  let strNum = "123";
  let num = Number(strNum);

  document.getElementById("numberConstructorDemo").innerHTML
= `Converted number: ${num}`;
</script>

</body>
</html>
```

Output:-

Converted number: 123

23.3 isNaN() Function

The isNaN() function checks if a value is **NaN (Not-a-Number)**.

Syntax:

```
isNaN(value)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>isNaN() Example</title>
</head>
<body>
<h2>isNaN() Example</h2>
<p id="isNaNDemo"></p>
```

```
<script>
  let value1 = 123;
  let value2 = "Hello";
  let result1 = isNaN(value1);
  let result2 = isNaN(value2);

  document.getElementById("isNaNDemo").innerHTML = `Is ${value1}
NaN? ${result1}<br>Is "${value2}" NaN? ${result2}`;
</script>
</body>
</html>
```

Output:-

Is 123 NaN? false
Is "Hello" NaN? true

23.4 parseInt() Function

The parseInt() function parses a string and returns an integer.

Syntax:

```
parseInt(string, radix)
```

Practical:-

```
<!DOCTYPE html>
<html>
<head>
  <title>parseInt() Example</title>
</head>
<body>
<h2>parseInt() Example</h2>
<p id="parseIntDemo"></p>

<script>
  let str = "42 years old";
  let age = parseInt(str);

  document.getElementById("parseIntDemo").innerHTML = `Parsed
integer: ${age}`;
</script>
</body>
</html>
```

Output:-

Parsed integer: 42

23.5 parseFloat() Function

The parseFloat() function parses a string and returns a floating-point number.

Syntax:

```
parseFloat(string)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>parseFloat() Example</title>
</head>
<body>

<h2>parseFloat() Example</h2>
<p id="parseFloatDemo"></p>

<script>
  let str = "3.14 is Pi";
  let piValue = parseFloat(str);

  document.getElementById("parseFloatDemo").innerHTML = `Parsed float: ${piValue}`;
</script>

</body>
</html>
```

Output:-

Parsed float: 3.14

23.6 toFixed() Method

The toFixed() method formats a number using **fixed-point** notation.

Syntax:

```
number.toFixed(digits)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>toFixed() Example</title>
</head>
<body>

<h2>toFixed() Example</h2>
<p id="toFixedDemo"></p>
<script>
  let number = 5.56789;
  let formattedNumber = number.toFixed(2);

  document.getElementById("toFixedDemo").innerHTML = `Formatted
number: ${formattedNumber}`;
</script>

</body>
</html>
```

Output:-

Formatted number: 5.57

24) JavaScript BigInt

24.1 Introduction to BigInt

BigInt is a special numeric type in JavaScript that allows you to represent and work with very large integers that can't be represented by the regular Number type.

Syntax:

```
BigInt(value)
```

Example:

```
let bigIntValue = 1234567890123456789012345678901234567890n;

console.log(bigIntValue);
```

Note: A BigInt is created by appending an n to the end of an integer or using the BigInt() constructor.

24.2 Using BigInt with Large Numbers

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>BigInt Example</title>
</head>
<body>

<h2>BigInt Example</h2>
<p id="bigIntDemo"></p>

<script>
  let largeNumber = 9007199254740991n;  // Bigger than the maximum safe integer
  let result = largeNumber + 1n;

  document.getElementById("bigIntDemo").innerHTML = `Large Number + 1 = ${result}`;
</script>

</body>
</html>
```

Output:-

Large Number + 1 = 9007199254740992

24.3 BigInt() Constructor

The BigInt() function can be used to convert primitive values into BigInt types.

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>BigInt Constructor Example</title>
</head>
<body>

<h2>BigInt Constructor Example</h2>
<p id="bigIntConstructorDemo"></p>
```

```
<script>
  let num = 12345;
  let bigIntValue = BigInt(num);
  document.getElementById("bigIntConstructorDemo").innerHTML =
`BigInt from number: ${bigIntValue}`;
</script>
</body>
</html>
```

Output:-

BigInt from number: 12345n

24.4 Operations with BigInt

You can perform arithmetic operations with BigInt just like with regular numbers, but both operands must be BigInt.

Example:

```
let bigInt1 = 10000000000000000000n;

let bigInt2 = 20000000000000000000n;

let sum = bigInt1 + bigInt2;

console.log(sum); // 30000000000000000000n
```

24.5 Limitations of BigInt

- **BigInt** cannot be mixed with regular numbers (Number type). You must use **all BigInts** or **all numbers** in an operation.
- **BigInt** doesn't support methods like .toFixed(), .toPrecision(), or .toExponential() which are available for regular numbers.

Example of Error:

```
let bigIntValue = 12345678901234567890123456789012345678901234567890n;

let regularNumber = 100;

console.log(bigIntValue + regularNumber);  // TypeError: Cannot mix BigInt and other types
```

25) JavaScript Number Methods

25.1 Number.isFinite()

The Number.isFinite() method determines whether the provided value is a **finite** number. It returns false for NaN, Infinity, or -Infinity.

Syntax:

```
Number.isFinite(value)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.isFinite() Example</title>
</head>
<body>
<h2>Number.isFinite() Example</h2>
<p id="isFiniteDemo"></p>

<script>
  let value1 = 42;
  let value2 = Infinity;

  let result1 = Number.isFinite(value1);
  let result2 = Number.isFinite(value2);

  document.getElementById("isFiniteDemo").innerHTML = `Is
${value1} finite? ${result1}<br>Is ${value2} finite? ${result2}`;
</script>

</body>
</html>
```

Output:-

Is 42 finite? true
Is Infinity finite? false

25.2 Number.isInteger()

The Number.isInteger() method checks if the value is an **integer** (i.e., not a decimal or a non-numeric value).

Syntax:

```
Number.isInteger(value)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.isInteger() Example</title>
</head>
<body>

<h2>Number.isInteger() Example</h2>
<p id="isIntegerDemo"></p>

<script>
  let value1 = 25;
  let value2 = 25.5;

  let result1 = Number.isInteger(value1);
  let result2 = Number.isInteger(value2);

  document.getElementById("isIntegerDemo").innerHTML = `Is
${value1} an integer? ${result1}<br>Is ${value2} an integer?
${result2}`;
</script>
</body>
</html>
```

Output:-

Is 25 an integer? true
Is 25.5 an integer? false

25.3 Number.parseInt()

The Number.parseInt() function parses a string argument and returns an **integer**.

Syntax:

```
Number.parseInt(string, radix)
```

Practical:

```
<!DOCTYPE html><html>
<head>
  <title>Number.parseInt() Example</title>
</head>
<body>
<h2>Number.parseInt() Example</h2>
<p id="parseIntDemo"></p>
<script>
  let str = "100 years";
  let parsedValue = Number.parseInt(str);
  document.getElementById("parseIntDemo").innerHTML = `Parsed
integer: ${parsedValue}`;
</script>
</body></html>
```

Output:-

Parsed integer: 100

25.4 Number.parseFloat()

The Number.parseFloat() function parses a string and returns a **floating-point** number.

Syntax:

```
Number.parseFloat(string)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.parseFloat() Example</title>
</head>
<body>
<h2>Number.parseFloat() Example</h2>
<p id="parseFloatDemo"></p>

<script>
  let str = "3.14159 is Pi";
  let parsedValue = Number.parseFloat(str);

  document.getElementById("parseFloatDemo").innerHTML = `Parsed
```

Output:-

Parsed float:

```
float: ${parsedValue}`;
</script>
</body></html>
```

25.5 Number.toFixed()

The toFixed() method formats a number to **a specified number of decimal places**.

Syntax:

```
number.toFixed(digits)
```

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.toFixed() Example</title>
</head>
<body>
<h2>Number.toFixed() Example</h2>
<p id="toFixedDemo"></p>

<script>
  let value = 3.14159;
  let formattedValue = value.toFixed(2);  // rounding to 2 decimal
places

  document.getElementById("toFixedDemo").innerHTML = `Formatted
number: ${formattedValue}`;
</script>
</body>
</html>
```

Output:-

Formatted number: 3.14

26) JavaScript Number Properties

26.1 Number.MAX_VALUE

Number.MAX_VALUE is the largest **finite** number that can be represented in JavaScript. Anything larger is considered Infinity.

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.MAX_VALUE Example</title>
</head>
<body>
<h2>Number.MAX_VALUE Example</h2>
<p id="maxValueDemo"></p>
<script>
  document.getElementById("maxValueDemo").innerHTML = `The
maximum value that JavaScript can represent is:
${Number.MAX_VALUE}`;
</script>

</body>
</html>
```

Output:-

The maximum value that JavaScript can represent is: 1.7976931348623157e+308

26.2 Number.MIN_VALUE

Number.MIN_VALUE is the smallest **positive** number greater than 0 that can be represented in JavaScript.

Practical:

```
<!DOCTYPE html><html>
<head>
  <title>Number.MIN_VALUE Example</title>
</head>
<body>
<h2>Number.MIN_VALUE Example</h2>
<p id="minValueDemo"></p>
<script>
  document.getElementById("minValueDemo").innerHTML = `The
smallest positive number in JavaScript is: ${Number.MIN_VALUE}`;
</script>
</body>
</html>
```

Output:-

The smallest positive number in JavaScript is: 5e-324

26.3 Number.NaN

Number.NaN represents the **Not-a-Number** value, typically resulting from an invalid number operation.

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.NaN Example</title>
</head>
<body>

<h2>Number.NaN Example</h2>
<p id="nanDemo"></p>

<script>
  let invalidNumber = 0 / 0;  // NaN result
  document.getElementById("nanDemo").innerHTML = `
invalid operation is: ${invalidNumber}`;
</script>

</body>
</html>
```

Output:-

The result of invalid operation is: NaN

26.4 Number.POSITIVE_INFINITY

Number.POSITIVE_INFINITY represents **positive infinity**. It is the result of a number exceeding Number.MAX_VALUE or dividing a positive number by 0.

Practical:

```
<!DOCTYPE html>
<html>
<head>
  <title>Number.POSITIVE_INFINITY Example</title>
</head>
<body>

<h2>Number.POSITIVE_INFINITY Example</h2>
```

```
<p id="posInfinityDemo"></p>

<script>
  let infinityValue = 1 / 0;  // Infinity result
  document.getElementById("posInfinityDemo").innerHTML = `The
result of dividing by zero is: ${infinityValue}`;
</script>
</body>
</html>
```

Output:-

The result of dividing by zero is: Infinity

26.5 Number.NEGATIVE_INFINITY

Number.NEGATIVE_INFINITY represents **negative infinity**, which is the result of dividing a negative number by 0.

Practical:

```
<!DOCTYPE html><html><head>
  <title>Number.NEGATIVE_INFINITY Example</title>
</head>
<body>
<h2>Number.NEGATIVE_INFINITY Example</h2>
<p id="negInfinityDemo"></p>
<script>
  let negativeInfinity = -1 / 0;  // Negative Infinity result
  document.getElementById("negInfinityDemo").innerHTML = `The
result of dividing by zero is: ${negativeInfinity}`;
</script>
</body></html>
```

Output:-

The result of dividing by zero is: -Infinity

Summary of Key JavaScript Number Properties:

- **MAX_VALUE**: The largest number JavaScript can represent.
- **MIN_VALUE**: The smallest positive number greater than zero.
- **NaN**: Not-a-Number.
- **POSITIVE_INFINITY**: Positive infinity.
- **NEGATIVE_INFINITY**: Negative infinity.

27) JavaScript Arrays

A JavaScript array is a single variable used to store multiple elements. It is a list-like object with numeric indexing.

```
let fruits = ["Apple", "Banana", "Cherry"];
```

27.1 Array Properties

Property	Description	Example
.length	Returns number of elements	fruits.length = 3
constructor	Returns the function that created array	fruits.constructor
prototype	Allows adding properties/methods	Array.prototype.myFunc = …

Practical – Array .length

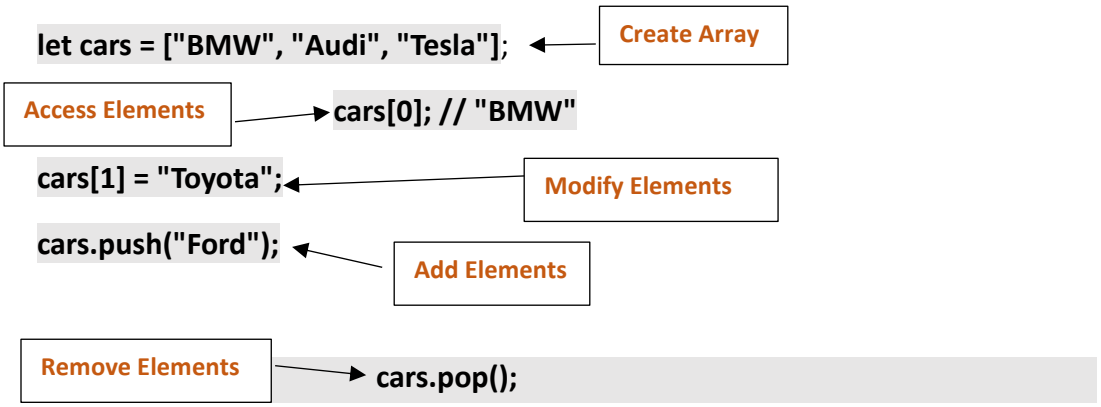
```
<!DOCTYPE html>
<html>
<head><title>Array Length</title></head>
<body>

<h3>Array Length</h3>
<p id="demo1"></p>

<script>
  let colors = ["Red", "Blue", "Green"];
  document.getElementById("demo1").innerHTML = "Length: " +
colors.length;
</script>

</body>
</html>
```

27.2 Basic Array Operations



27.3 Common Array Methods

Method	Description
push()	Add item to end
pop()	Remove item from end
shift()	Remove item from start
unshift()	Add item to start
concat()	Merge arrays
join()	Join elements as string
slice()	Extract section
splice()	Add/remove elements
indexOf()	First index of item
lastIndexOf()	Last index of item
includes()	Check if item exists
reverse()	Reverse array
sort()	Sort elements

Method	Description
toString()	Convert to string
fill()	Fill with static values
flat()	Flatten nested array
find()	Return first match
findIndex()	Return index of first match

27.3.1 push() – Add to end

```
<script>
  let numbers = [1, 2, 3];
  numbers.push(4); // [1, 2, 3, 4]
</script>
```

27.3.2 pop() – Remove last

```
<script>
  let numbers = [1, 2, 3];
  numbers.pop(); // [1, 2]
</script>
```

27.3.3 shift() – Remove first

```
<script>
  let names = ["Alice", "Bob", "Eve"];
  names.shift(); // ["Bob", "Eve"]
</script>
```

27.3.4 unshift() – Add to start

```
<script>
  let names = ["Bob", "Eve"];
  names.unshift("Alice"); // ["Alice", "Bob", "Eve"]
</script>
```

27.3.5 concat() – Merge arrays

```
<script>
  let a = [1, 2];
  let b = [3, 4];  let combined = a.concat(b); // [1, 2, 3, 4]
</script>
```

27.3.6 join() – Convert to string

```
<script>
  let fruits = ["Apple", "Banana"];
  let result = fruits.join(" - "); // "Apple - Banana"
</script>
```

27.3.7 slice() – Copy part of array

```
<script>
  let fruits = ["Apple", "Banana", "Cherry"];
  let sliced = fruits.slice(1, 3); // ["Banana", "Cherry"]
</script>
```

27.3.8 splice() – Add/remove items

```
<script>
  let arr = [1, 2, 3];
  arr.splice(1, 0, 5); // [1, 5, 2, 3]
</script>
```

27.3.9 indexOf() – Find index

```
<script>
  let nums = [5, 10, 15];
  let pos = nums.indexOf(10); // 1
</script>
```

27.3.10 lastIndexOf()

```
<script>
  let vals = [1, 2, 3, 2];
  let idx = vals.lastIndexOf(2); // 3
</script>
```

27.3.11 includes()

```
<script>
  let items = ["a", "b", "c"];
  let check = items.includes("b"); // true
</script>
```

27.3.12 reverse()

```
<script>
  let arr = [1, 2, 3];
  arr.reverse(); // [3, 2, 1]
</script>
```

27.3.13 sort()

```
<script>
  let nums = [5, 1, 3];
  nums.sort(); // [1, 3, 5]
</script>
```

27.3.14 toString()

```
<script>
  let arr = [1, 2, 3];
  let str = arr.toString(); // "1,2,3"
</script>
```

27.3.15 fill()

```
<script>
  let arr = new Array(3).fill(0); // [0, 0, 0]
</script>
```

27.3.16 flat()

```
<script>
  let nested = [1, [2, 3]];
  let flatArr = nested.flat(); // [1, 2, 3]
</script>
```

27.3.17 find()

```
<script>
  let ages = [10, 20, 30];
  let adult = ages.find(age => age >= 18); // 20
</script>
```

27.3.18 findIndex()

```
<script>
  let scores = [45, 50, 90];
  let index = scores.findIndex(score => score > 60); // 2
</script>
```

27.4 Looping Over Arrays

27.4.1 For loop:

```
for (let i = 0; i < arr.length; i++) { console.log(arr[i]); }
```

27.4.2 For...of loop:

```
for (let item of arr) { console.log(item); }
```

27.4.3 ForEach() method:

```
arr.forEach(function(item) { console.log(item); });
```

Practical Demo

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Arrays Demo</title></head>
<body>
<h2>Array Methods Demo</h2>
<p id="arrayOutput"></p>

<script>
  let fruits = ["Apple", "Banana", "Cherry"];
  fruits.push("Mango");
  fruits.splice(1, 1, "Orange"); // Replace Banana
```

```
  let result = fruits.join(" - ");
  document.getElementById("arrayOutput").innerHTML = "Fruits: " +
result;
</script>
</body>
</html>
```

Summary

- Arrays are zero-indexed, resizable, and dynamic.
- JavaScript provides 20+ useful methods.
- Arrays can hold any data type, including other arrays (nested).

27.5 Practice Questions based on Array

Q.27.5.1 What will be the output of the following code, and why?

```
let numbers = [1, 2, 3, 4, 5];
numbers.forEach((num, index) => {
  if (num % 2 === 0) {
    numbers.splice(index, 1);
  }
});
console.log(numbers);
```

Options:

A) [1, 3, 5]
B) [1, 2, 3, 4, 5]
C) [1, 3, 4, 5]
D) [1, 3, 5] but with an error

Explanation:

- The forEach() loop **does not create a copy** of the array. It works on the **original array**.
- When splice() removes an element, the array's length changes **and shifts** the remaining elements **left**.
- After removing an element, the next index gets skipped because the loop moves to the next index while the array has already shifted.

Step-by-step:

1. num = 1 → not even → skip
2. num = 2 at index 1 → even → remove 2 → numbers = [1, 3, 4, 5]
3. Next index is 2, which is now 4, so 3 is skipped!
4. num = 4 at index 2 → even → remove → numbers = [1, 3, 5]
5. Done

But because of skipping and index shifts, the actual **printed output is [1, 3, 4, 5]**.

Question 27.5.2: Predict the output for following code ?

```
let result = [1, 2, 3].map(num => {
  num * 2;
});
console.log(result);
```

Options:

A) [2, 4, 6]
B) [undefined, undefined, undefined]
C) [1, 2, 3]
D) Throws error

Explanation: The map() method expects a return value. Since num * 2 isn't returned explicitly, each iteration returns undefined.

Question 27.5.3: Array Mutation Inside Loop

```
let arr = [10, 20, 30, 40];
for (let i = 0; i < arr.length; i++) {
  arr.pop();
}
console.log(arr);
```

Options:

A) []
B) [10, 20] C) [10, 20, 30, 40]
D) [10, 20, 30]

Explanation:

- Loop runs while i < arr.length.
- On first loop: arr.length = 4, pop() removes 40.
- On second loop: arr.length = 3, pop() removes 30.
- i = 2, arr.length = 2 → loop ends.

Only two elements remain.

Question 27.5.4: splice() Behavior

```
let colors = ["red", "blue", "green"];
colors.splice(1, 0, "yellow", "purple");
console.log(colors);
```

Options:

A) ["red", "yellow", "purple", "blue", "green"]

B) ["red", "blue", "green", "yellow", "purple"]

C) ["red", "yellow", "blue", "green", "purple"]

D) ["yellow", "purple", "red", "blue", "green"]

Explanation:

.splice(1, 0, "yellow", "purple") inserts "yellow" and "purple" **at index 1**, deleting **0** items.

Question 27.5.5: filter() and Truthy Values

```
let arr = [0, 1, 2, "", false, 3];
let filtered = arr.filter(Boolean);
console.log(filtered);
```

Options:

A) [1, 2, 3]

B) [0, 1, 2, "", false, 3]

C) [1, 2, "", false, 3]

D) [1, 2, 3, 0, ""]

Explanation:

filter(Boolean) removes all **falsy** values: 0, "", false, null, undefined, and NaN.

Question 27.5.6: reduce() Trick

```
let sum = [1, 2, 3, 4].reduce((acc, val) => acc + val, 5);
console.log(sum);
```

Options:

A) 10

B) 15

C) 5

D) None

Explanation: The initial accumulator value is 5. Then it adds all elements:
5 + 1 + 2 + 3 + 4 = 15.

Q: 27.6:- Coding Challenge: Filter and Display Students

Problem Statement:
You are given an array of student objects. Each student has the following properties:

- name (string)
- marks (number)

Write a complete HTML + JavaScript program that:

1. Displays the list of **all students** in an HTML table.
2. Filters and displays only those students who scored **more than 75 marks** in a **separate table**.
3. Uses **Array methods (filter, forEach) and loops** appropriately.

Bonus: Add a button that, when clicked, filters and displays only the top scorers.

Sample Input Array:

```
let students = [
 { name: "Amit", marks: 82 },
 { name: "Priya", marks: 68 },
 { name: "Ravi", marks: 91 },
 { name: "Sneha", marks: 73 },
 { name: "John", marks: 77 } ];
```

Expected Output:

- A full HTML page with:
    - Table 1: All students
    - Table 2 (after button click): Only students with marks > 75

Full Solution Code

```
<!DOCTYPE html>
<html>
<head>
 <title>Student Marks Filter</title>
 <style>
  table {
   width: 50%;
   border-collapse: collapse;
   margin-bottom: 20px;
  }
  th, td {
   padding: 8px;
   border: 1px solid #444;
   text-align: left;
  }
  th {
   background-color: #f2f2f2;
  }
  button {
   padding: 10px 15px;
   font-size: 16px;
   cursor: pointer;
  }
 </style>
</head>
<body>
 <h2>All Students</h2>
 <table id="allStudentsTable">
  <thead>
   <tr>
    <th>Name</th>
    <th>Marks</th>
   </tr>
  </thead>
```

```
  <tbody>
    <!-- All students will be inserted here -->
  </tbody>
 </table>

 <button onclick="showTopScorers()">Show Top Scorers (Marks > 75)</button>
 <h2>Top Scorers</h2>
 <table id="topScorersTable">
  <thead>
    <tr>
      <th>Name</th>
      <th>Marks</th>
    </tr>
  </thead>
  <tbody>
    <!-- Top scorers will be inserted here -->
  </tbody>
 </table>

 <script>
  let students = [
    { name: "Amit", marks: 82 },
    { name: "Priya", marks: 68 },
    { name: "Ravi", marks: 91 },
    { name: "Sneha", marks: 73 },
    { name: "John", marks: 77 }
  ];

  // Display all students in the first table
  function displayAllStudents() {
    const tableBody = document.querySelector("#allStudentsTable tbody");
    tableBody.innerHTML = "";

    students.forEach(student => {
      const row = `<tr><td>${student.name}</td><td>${student.marks}</td></tr>`;
      tableBody.innerHTML += row;
    });
  }
```

```
  // Filter and show top scorers (marks > 75)
  function showTopScorers() {
    const topScorers = students.filter(s => s.marks > 75);
    const tableBody = document.querySelector("#topScorersTable
tbody");
    tableBody.innerHTML = "";

    topScorers.forEach(student => {
      const row =
`<tr><td>${student.name}</td><td>${student.marks}</td></tr>`;
      tableBody.innerHTML += row;
    });
  }

  // Initial rendering of all students
  displayAllStudents();
 </script>
</body>
</html>
```

28) JavaScript Dates

JavaScript uses the built-in Date object to work with dates and times. It allows you to create, manipulate, and format date values.

Ways to Create a Date Object:

```
let date1 = new Date(); // current date and time

let date2 = new Date("2024-05-10"); // specific date (YYYY-MM-DD)

let date3 = new Date(2024, 4, 10); // year, month (0-indexed), day

let date4 = new Date(2024, 4, 10, 14, 30); // year, month, day, hour, minute
```

Use Case:

Creating logs, timestamps, scheduling events, deadlines, or birthdays.

28.1 JavaScript Dates Format

Built-in Date Formatting Methods:

Method	Description	Example Output (May 10, 2024 at 14:30)
toDateString()	Converts to a readable date	Fri May 10 2024
toTimeString()	Converts to a readable time	14:30:00 GMT+0530 (IST)
toISOString()	Returns date in ISO format	2024-05-10T09:00:00.000Z
toLocaleDateString()	Localized date string	5/10/2024 (in US format)
toLocaleTimeString()	Localized time string	2:30:00 PM

Example:

```
let now = new Date();
console.log(now.toDateString());
console.log(now.toTimeString());
console.log(now.toISOString());
console.log(now.toLocaleDateString());
console.log(now.toLocaleTimeString());
```

28.2) JavaScript Dates Get Methods

To **extract** information from a Date object.

Common Get Methods:

```
let now = new Date();

now.getFullYear();     // 2024

now.getMonth();       // 4 (May; 0-based)

now.getDate();        // 10

now.getDay();         // 5 (Friday; 0 = Sunday)

now.getHours();       // 14

now.getMinutes();     // 30
```

```
now.getSeconds();      // 0
now.getMilliseconds(); // 0
now.getTime();         // Milliseconds since Jan 1, 1970
```

Use Case:

Used in clocks, birthday reminders, calendar views, etc.

28.3 JavaScript Dates Set Methods

To **modify** parts of an existing Date object.

Common Set Methods:

```
let date = new Date();
date.setFullYear(2025);    // Change year to 2025
date.setMonth(0);          // January
date.setDate(15);          // 15th
date.setHours(10);         // 10 AM
date.setMinutes(45);       // 45 minutes
date.setSeconds(30);       // 30 seconds
```

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Dates Demo</title>
  <style>
    body { font-family: Arial; }
    .box { padding: 15px; background: #f9f9f9; border: 1px solid #ccc;
margin: 10px 0; }
  </style>
</head>
<body>
<h2>JavaScript Date Demo</h2>
<div class="box" id="output"></div>
```

```
<script>
  let now = new Date();
  let customDate = new Date(2024, 4, 10, 14, 30); // May 10, 2024 2:30
PM
  // Get Methods
  let year = now.getFullYear();
  let month = now.getMonth() + 1; // add 1 because it's 0-based
  let date = now.getDate();
  let day = now.getDay();
  let hours = now.getHours();
  let minutes = now.getMinutes();

  // Format
  let output = `
    <strong>Current Date:</strong> ${now.toDateString()}<br>
    <strong>Time:</strong> ${now.toTimeString()}<br>
    <strong>ISO Format:</strong> ${now.toISOString()}<br>
    <strong>Localized Date:</strong> ${now.toLocaleDateString()}<br>
    <strong>Localized Time:</strong>
${now.toLocaleTimeString()}<br><br>

    <strong>GET Methods:</strong><br>
    Year: ${year}<br>
    Month: ${month}<br>
    Day of Month: ${date}<br>
    Day of Week: ${day} (0=Sunday)<br>
    Hours: ${hours}<br>
    Minutes: ${minutes}<br><br>

    <strong>Custom Date Set (May 10, 2024, 2:30PM):</strong><br>
    ${customDate};
  document.getElementById("output").innerHTML = output;
</script>

</body>
</html>
```

Output (in browser):

Current Date: Fri May 10 2024

Time: 14:30:00 GMT+0530 (India Standard Time)

ISO Format: 2024-05-10T09:00:00.000Z

Localized Date: 5/10/2024

Localized Time: 2:30:00 PM

GET Methods:

Year: 2024

Month: 5

Day of Month: 10

Day of Week: 5 (0=Sunday)

Hours: 14

Minutes: 30

Custom Date Set (May 10, 2024, 2:30PM):

Fri May 10 2024 14:30:00 GMT+0530 (India Standard Time)

28.4 Real World Use Case: Age Calculator

Problem: Calculate a person's current age based on their birthdate.

Code:

```
<!DOCTYPE html>
<html>
<head><title>Age Calculator</title></head>
<body>
<h3>Enter your birthdate:</h3>
<input type="date" id="birthdate">
<button onclick="calculateAge()">Calculate Age</button>
<p id="ageOutput"></p>
<script>
function calculateAge() {
  let birth = new Date(document.getElementById("birthdate").value);
```

```
  let today = new Date();

  let age = today.getFullYear() - birth.getFullYear();
  let m = today.getMonth() - birth.getMonth();

  if (m < 0 || (m === 0 && today.getDate() < birth.getDate())) {
    age--;
  }

  document.getElementById("ageOutput").innerText = "You are " +
age + " years old.";
}
</script>
</body>
</html>
```

Edge Cases

- Month is 0-indexed, so January = 0, December = 11.
- Invalid date strings produce Invalid Date.
- Time zones affect string outputs like toTimeString().

29) JavaScript Math

- Math is a **static built-in object**.
- It provides **mathematical constants and functions**.
- Accessed directly via Math.method() or Math.property.

You **cannot** use new Math() – it's **not a constructor**.

29.1 Math Constants (Properties)

Property	Description	Value
Math.PI	Ratio of circumference to diameter	3.141592653589793
Math.E	Euler's number	2.718281828459045
Math.LN2	Natural log of 2	0.6931471805599453

Property	Description	Value
Math.LN10	Natural log of 10	2.302585092994046
Math.LOG2E	Log base 2 of E	1.4426950408889634
Math.LOG10E	Log base 10 of E	0.4342944819032518
Math.SQRT1_2	Square root of 1/2	0.7071067811865476
Math.SQRT2	Square root of 2	1.4142135623730951

29.2 Math Methods

29.2.1. Math.round(x):- Round a number to the nearest integer. Used in Price rounding, salary approximation.

```
Math.round(4.4); // 4

Math.round(4.5); // 5
```

29.2.2. Math.ceil(x):- Round number UP to nearest integer. Used in Ticketing system, billing, step increments.

```
Math.ceil(4.1); // 5
```

29.2.3. Math.floor(x):- Round number DOWN to nearest integer. Used in Age truncation, integer division.

```
Math.floor(4.9); // 4
```

29.2.4. Math.trunc(x):- Remove decimal (no rounding).used in Drop cents in billing.

```
Math.trunc(9.81); // 9
```

29.2.5. Math.abs(x):- Return absolute value.used in Distance, error difference calculations.

```
Math.abs(-10); // 10
```

29.2.6. Math.pow(base, exponent):- Physics calculations, compound interest.

```
Math.pow(2, 3); // 8
```

29.2.7. Math.sqrt(x):- Square root. Used in Geometry, statistics (standard deviation).

```
Math.sqrt(64); // 8
```

29.2.8. Math.min(a, b, c...):- Return the smallest number.Used in finding lowest score, minimum price.

```
Math.min(5, 10, 2); // 2
```

29.2.9. Math.max(a, b, c...):- Return the largest number. Used in High score, max profit.

```
Math.max(5, 10, 2); // 10
```

29.2.10. Math.random():- Returns random decimal from 0 (inclusive) to 1(exclusive). Used in Games, OTP, password generators.

```
Math.random(); // 0.382918291...
```

29.2.11. Math.log(x):- Natural logarithm (base E). Used in Scientific calculations.

```
Math.log(1); // 0
Math.log(Math.E); // 1
```

29.2.12. Math.sin(x) / Math.cos(x) / Math.tan(x):- Trigonometric functions (in radians). Used in Animations, engineering apps.

```
Math.sin(Math.PI / 2); // 1
Math.cos(0); // 1
```

Practical Example

```
<!DOCTYPE html><html>
<head><title>JavaScript Math Demo</title>
 <style>
  body { font-family: Arial; background: #f9f9f9; padding: 20px; }
```

```
   .result { background: #eef; padding: 15px; margin-top: 10px; border-left: 4px solid #2196F3; }
  </style></head>
<body>
<div class="result" id="output"></div>
<script>
  let result = `
    <strong>Math.round(4.6):</strong> ${Math.round(4.6)} <br>
    <strong>Math.ceil(4.1):</strong> ${Math.ceil(4.1)} <br>
    <strong>Math.floor(4.9):</strong> ${Math.floor(4.9)} <br>
    <strong>Math.trunc(9.8):</strong> ${Math.trunc(9.8)} <br>
    <strong>Math.abs(-12):</strong> ${Math.abs(-12)} <br>
    <strong>Math.pow(2, 3):</strong> ${Math.pow(2, 3)} <br>
    <strong>Math.sqrt(36):</strong> ${Math.sqrt(36)} <br>
    <strong>Math.min(3, 7, 2):</strong> ${Math.min(3, 7, 2)} <br>
    <strong>Math.max(3, 7, 2):</strong> ${Math.max(3, 7, 2)} <br>
    <strong>Math.PI:</strong> ${Math.PI} <br>
    <strong>Math.E:</strong> ${Math.E} <br>
    <strong>Math.random():</strong> ${Math.random()} <br>
    <strong>Math.sin(Math.PI / 2):</strong> ${Math.sin(Math.PI / 2)} <br> `;
  document.getElementById("output").innerHTML = result;
</script>
</body></html>
```

Expected Output:-

Math.round(4.6): 5

Math.ceil(4.1): 5

Math.floor(4.9): 4

Math.trunc(9.8): 9

Math.abs(-12): 12

Math.pow(2, 3): 8

Math.sqrt(36): 6

Math.min(3, 7, 2): 2

Math.max(3, 7, 2): 7

Math.PI: 3.141592653589793

Math.E: 2.718281828459045

Math.random(): 0.5479281 (varies)

Math.sin(Math.PI / 2): 1

30) JavaScript Random

Math.random() is a method of the Math object.It **returns a floating-point, pseudo-random number** in the **range [0, 1)**. It includes 0,It **never** returns 1

It is Useful for creating randomness in games, OTPs, passwords, simulations, random selections, etc.

Syntax:

```
Math.random()
```

Practical Use Case:

Use Case	Example
Random OTP generation	4-digit PIN
Dice game	Random number from 1–6
Coin flip	Heads/Tails using 0 or 1
Lottery, game mechanics	Random selection from array
Random background color	Use RGB values
Shuffling items	Randomize an array

30.1 Custom Random Ranges

30.1.1 Random Integer Between 0 and N-1

```
let random0to9 = Math.floor(Math.random() * 10);
console.log(random0to9); // 0 to 9
```

30.1.2 Random Integer Between 1 and N

```
let random1to10 = Math.floor(Math.random() * 10) + 1;
console.log(random1to10); // 1 to 10
```

30.1.3 Random Integer Between min and max (inclusive)

```
function getRandomInRange(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(getRandomInRange(20, 30)); // 20 to 30
```

30.1.4 Bonus Use Case: Dice Game

```
function rollDice() {
  return Math.floor(Math.random() * 6) + 1;
}
console.log("Dice Rolled: " + rollDice());
```

Practical Demo

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Random Examples</title>
  <style>
    body { font-family: Arial; background-color: #f0f0f0; padding: 20px; }
```

```
   .result { background: #e0f7fa; border-left: 5px solid #00796B;
padding: 15px; margin: 10px 0; }
  </style>
</head>
<body>
<h2>JavaScript Random – Practical Examples</h2>
<div class="result" id="output"></div>

<script>
  let output = `
    <strong>Random Decimal (0 to 1):</strong> ${Math.random()} <br>
    <strong>Random Integer (0 to 9):</strong>
${Math.floor(Math.random() * 10)} <br>
    <strong>Random Integer (1 to 10):</strong>
${Math.floor(Math.random() * 10) + 1} <br>
    <strong>Random Integer (20 to 30):</strong>
${Math.floor(Math.random() * (30 - 20 + 1)) + 20} <br>
    <strong> Dice Roll (1 to 6):</strong> ${Math.floor(Math.random() *
6) + 1} <br>
    <strong>Coin Toss (0 = Heads, 1 = Tails):</strong>
${Math.floor(Math.random() * 2)} <br>
  `;

  document.getElementById("output").innerHTML = output;
</script>
</body>
</html>
```

Output (Varies Every Time)

Random Decimal (0 to 1): 0.3948721

Random Integer (0 to 9): 6

Random Integer (1 to 10): 3

Random Integer (20 to 30): 24

Dice Roll (1 to 6): 5

Coin Toss (0 = Heads, 1 = Tails): 1

31) JavaScript Booleans

A **Boolean** in JavaScript represents one of **two values**: **True & False**

These are **primitive data types** used for logical operations and conditional decisions.

Syntax:

```
let isAvailable = true;

let isLoggedIn = false;
```

31.1 Boolean as Result of Comparison

Boolean values often come from **comparison operators**:

```
console.log(10 > 5);   // true

console.log(3 === 4);  // false
```

Value	Type
false	Boolean
0, -0	Number
""	String
null	Null
undefined	Undefined
NaN	Number

31.2 Boolean Constructor

```
let bool1 = Boolean(10);      // true

let bool2 = Boolean(0);       // false

let bool3 = Boolean("Hello");  // true

let bool4 = Boolean("");      // false
```

31.3 Use Cases of Boolean

Use Case	Example
Login/Authentication	isLoggedIn = true
Form validation	isValid = false
Feature toggle	darkMode = true
Decision making	if (isAvailable)
Loop control	while (isRunning)

Practical Example 1 :-

```
<!DOCTYPE html><html>
<head>
  <title>JavaScript Booleans</title>
  <style>
    body { font-family: Arial; padding: 20px; background: #f0f8ff; }
    .output { background: #e3f2fd; padding: 15px; border-left: 5px solid
#0288d1; margin: 10px 0; }
  </style>
</head>
<body>
<h2>JavaScript Booleans – Practical Examples</h2>
<div class="output" id="output"></div>
<script>
  let output = "";
  let a = Boolean(100);    // true
  let b = Boolean(0);      // false
  let c = Boolean("");     // false
  let d = Boolean("Hello");// true
  let e = Boolean([]);     // true
  let f = Boolean(null);   // false
  output += `<strong>Boolean(100):</strong> ${a} <br>`;
  output += `<strong>Boolean(0):</strong> ${b} <br>`;
  output += `<strong>Boolean(""):</strong> ${c} <br>`;
  output += `<strong>Boolean("Hello"):</strong> ${d} <br>`;
  output += `<strong>Boolean([]):</strong> ${e} <br>`;
```

```
  output += `<strong>Boolean(null):</strong> ${f} <br>`;

  // Boolean from Comparison
  output += `<br><strong>10 > 5:</strong> ${10 > 5} <br>`;
  output += `<strong>10 === "10":</strong> ${10 === "10"} <br>`;
  output += `<strong>typeof true:</strong> ${typeof true} <br>`;

  document.getElementById("output").innerHTML = output;
</script>
</body>
</html>
```

Output (in browser):

Boolean(100): true

Boolean(0): false

Boolean(""): false

Boolean("Hello"): true

Boolean([]): true

Boolean(null): false

10 > 5: true

10 === "10": false

typeof true: boolean

Practical Example 2 : Login Simulation

```
let isLoggedIn = true;
if (isLoggedIn) {
  console.log("Welcome back, user!");
} else {
  console.log("Please log in.");
}
```

Output: Welcome back, user!

31.4 Quiz Question

Q.1 : What will be the result of this code?

```
let x = "";
if (x) {
  console.log("Truthy");
} else {
  console.log("Falsy");
}
```

Correct Answer: Falsy – because "" is a falsy value.

32) JavaScript Comparisons

Comparison operators **compare two values** and return a **Boolean** result: true or false.They are commonly used in if, while, for, and other control structures.

32.1 Types of Comparison Operators

Operator	Name	Example	Result
==	Equal to	5 == "5"	true
===	Equal value and type	5 === "5"	false
!=	Not equal	5 != 4	true
!==	Not equal value or type	5 !== "5"	true
>	Greater than	6 > 5	true
<	Less than	3 < 5	true
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	4 <= 3	false

32.2 Difference between (== & ===)

- == performs **type Checking**. 5 == "5" **// true**
- === checks **both value and type** (recommended). 5 === "5" **// false**

32.3 Comparison with Strings

Strings are compared **lexicographically (dictionary order)**:

console.log("apple" < "banana"); **// true**

console.log("2" > "12"); **// true ("2" > "1")**

32.3 Comparison with Booleans and Null

console.log(true == 1); **// true**

console.log(false == 0); **// true**

console.log(null == undefined**);** **// true**

console.log(null === undefined); **// false**

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Comparison Operators</title>
  <style>
    body { font-family: Arial; background: #fff8e1; padding: 20px; }
    .result { background: #fffde7; border-left: 5px solid #fbc02d; padding:
15px; margin: 10px 0; }
  </style>
</head>
<body>
<h2>JavaScript Comparison Operators – Practical Output</h2>
<div class="result" id="output"></div>
<script>
  let output = "";
```

```
  output += `<strong>5 == "5":</strong> ${5 == "5"} <br>`;
  output += `<strong>5 === "5":</strong> ${5 === "5"} <br>`;
  output += `<strong>5 != "5":</strong> ${5 != "5"} <br>`;
  output += `<strong>5 !== "5":</strong> ${5 !== "5"} <br>`;
  output += `<strong>6 > 5:</strong> ${6 > 5} <br>`;
  output += `<strong>3 < 5:</strong> ${3 < 5} <br>`;
  output += `<strong>5 >= 5:</strong> ${5 >= 5} <br>`;
  output += `<strong>4 <= 3:</strong> ${4 <= 3} <br>`;

  // String comparison
  output += `<br><strong>"apple" < "banana":</strong> ${"apple" <
"banana"} <br>`;
  output += `<strong>"2" > "12":</strong> ${"2" > "12"} <br>`;

  // Null, Boolean
  output += `<br><strong>true == 1:</strong> ${true == 1} <br>`;
  output += `<strong>false == 0:</strong> ${false == 0} <br>`;
  output += `<strong>null == undefined:</strong> ${null == undefined}
<br>`;
  output += `<strong>null === undefined:</strong> ${null ===
undefined} <br>`;

  document.getElementById("output").innerHTML = output;
</script>

</body>
</html>
```

Output (in browser):

5 == "5": true

5 === "5": false

5 != "5": false

5 !== "5": true

6 > 5: true

3 < 5: true

5 >= 5: true

4 <= 3: false

"apple" < "banana": true

"2" > "12": true

true == 1: true

false == 0: true

null == undefined: true

null === undefined: false

32.4 Quiz Questions

Question 1: What will this return?

```
console.log("5" === 5);
```

Answer: false (different types)

Question 2: What does this output?

```
console.log(null == undefined);
```

Answer: true (loosely equal)

Question 3 (Challenge):

```
if ("10" > 2) {
  console.log("Yes");
} else {
  console.log("No");
}
```

Answer: "Yes"
Explanation: "10" is coerced to number for numeric comparison.

33) JavaScript Hoisting

Hoisting is JavaScript's default behavior of **moving declarations to the top** of the current scope (either global or function scope) **before the code is executed**.

You can use variables and functions **before declaring them**, due to hoisting.

Item	Hoisted?	Initialized?	Notes
var variables	Yes	No	Hoisted but initialized as undefined
let / const	Yes	No	Hoisted to the **Temporal Dead Zone (TDZ)**
function decl.	Yes	Yes	Fully hoisted (can be called before declare)
class decl.	Yes	No	Hoisted but not initialized
function expressions	No	No	Not hoisted if using let, const, or var

Important Note:

- Only **declarations** are hoisted, not **initializations**.
- For let and const, hoisting puts them in the **Temporal Dead Zone (TDZ)** — accessing them before declaration throws a ReferenceError.

Example 1 – var hoisting

```
console.log(a); // undefined
var a = 10;
console.log(a); // 10
```

Explanation:

JavaScript interprets it like this:

```
var a;      // hoisted
console.log(a); // undefined
a = 10;
console.log(a); // 10
```

Example 2 – let and const hoisting

```
console.log(b); // ReferenceError

let b = 20;

console.log(c); // ReferenceError

const c = 30;
```

Explanation:
They are hoisted but not initialized → **TDZ error** when accessed before declaration.

Example 3 – Function Declaration Hoisting

```
greet(); // "Hello!"

function greet() {

  console.log("Hello!");

}
```

Explanation: Works perfectly — function declarations are **fully hoisted**.

Example 4 – Function Expression (not hoisted)

```
sayHi(); // TypeError: sayHi is not a function

var sayHi = function () {

  console.log("Hi!");

};
```

Explanation:
Only the variable sayHi is hoisted, but its value (the function) is not assigned at the time of access.

Use Case Example – Avoiding Bugs

```
function showLoginStatus() {

  if (isLoggedIn) {
```

```
  console.log("Welcome back!");
 } else {
  console.log("Please log in.");
 }
 var isLoggedIn = true;
}
```

Output:

Please log in.

Explanation:
Due to hoisting:

var isLoggedIn; // hoisted as undefined

if (isLoggedIn) // false

Practical :-

```
<!DOCTYPE html>
<html>
<head>
 <title>JavaScript Hoisting Demo</title>
 <style>
  body { font-family: Arial; background: #eef2f3; padding: 20px; }
  .box { background: #fff; border-left: 5px solid #2196f3; padding: 15px;
margin-bottom: 10px; }
 </style>
</head>
<body>

<h2>JavaScript Hoisting – Examples</h2>
<div class="box" id="output"></div>

<script>
 let out = "";

 // Example 1: var
 out += "<strong>var Example:</strong><br>";
 out += "Before declaration: " + a + "<br>";
 var a = 100;
```

```
  out += "After declaration: " + a + "<br><br>";

  // Example 2: function declaration
  out += "<strong>Function Declaration:</strong><br>";
  function test() {
   return "Function is hoisted!";
  }
  out += test() + "<br><br>";

  // Example 3: Function Expression
  out += "<strong>Function Expression:</strong><br>";
  try {
   out += callMe() + "<br>";
  } catch (e) {
   out += "Error: " + e.message + "<br>";
  }
  var callMe = function () {
   return "Function expression called";
  };
  out += "After declaration: " + callMe() + "<br><br>";

  // Example 4: let hoisting
  out += "<strong>let Hoisting:</strong><br>";
  try {
   out += b;
  } catch (e) {
   out += "Error: " + e.message + "<br>";
  }
  let b = 50;

  document.getElementById("output").innerHTML = out;
</script>

</body>
</html>
```

Output:

var Example:

Before declaration: undefined

After declaration: 100

Function Declaration:

Function is hoisted!

Function Expression:

Error: callMe is not a function

After declaration: Function expression called

let Hoisting:

Error: Cannot access 'b' before initialization

Quick Summary

Concept	var	let / const	function
Hoisted?	Yes	Yes	Yes
Initialized at hoist time?	No	No (TDZ)	Yes
Access before declaration	undefined	Error	OK

33.1 Quiz Time!

Q1: What is the output?

```
console.log(x);

var x = 5;
```

Answer: undefined

Q2: What happens here?

```
console.log(y);

let y = 10;
```

Answer: ReferenceError: Cannot access 'y' before initialization

Q3: What about this?

```
hello();
function hello() {
  console.log("Hello world");
}
```

Answer: Hello world

Q4: Write output

```
var test = "global";
function check() {
  console.log(test);
  var test = "local";
}
check();
```

Answer: undefined
(Explanation: var test inside the function is hoisted but not initialized)

34) JavaScript Arrow Functions

Arrow functions are a **shorter syntax** for writing function expressions in JavaScript. Introduced in **ES6 (ECMAScript 2015)**, they are especially useful for writing **concise and cleaner code**, especially for small operations and callbacks.

Syntax:-

```
const functionName = (parameters) => {
  // function body
};
```

34.1 Shorter version if there's only one expression:

```
const add = (a, b) => a + b;
```

34.2 Single parameter (no parentheses needed):

```
const greet = name => console.log("Hi", name);
```

Example 1 – Basic Arrow Function

```
const sayHello = () => {
  return "Hello World!";
};
console.log(sayHello());
```

Output: Hello World!

Example 2 – Implicit Return

```
const multiply = (a, b) => a * b;
console.log(multiply(4, 5));
```

Output: 20

Example 3 – One Parameter

```
const square = n => n * n;
console.log(square(6));
```

Output:36

Example 4 – No Parameters

```
const welcome = () => console.log("Welcome!");
welcome();
```

Output: Welcome!

Example 5 – Returning an Object **:-** To return an object **implicitly**, wrap it in parentheses:

```
const getUser = () => ({ name: "Alice", age: 25 });
console.log(getUser());
```

Output: { name: 'Alice', age: 25 }

34.3 Arrow Function vs Regular Function

Feature	Arrow Function	Regular Function
Short syntax	Yes	No
this binding	Inherits from parent	Own this
arguments object	Not available	Available
Constructors (new)	Not usable	Usable

Behavior of this in Arrow Functions

Arrow functions do **not** bind their own this. Instead, they inherit it from the **enclosing scope**.

```
const person = {
  name: "Alex",
  greet: function () {
    const inner = () => {
      console.log("Hi " + this.name);
    };
    inner();
  }
};

person.greet();
```

Output:Hi Alex

Arrow function captures this from the greet() method.

Common Mistake: Using Arrow Function in Methods

```
const person = {
 name: "Sam",
 greet: () => {
  console.log("Hello " + this.name);
 }
};

person.greet();
```

Output: Hello undefined

Reason: Arrow function doesn't have its own this.

Practical:-

```
<!DOCTYPE html>
<html>
<head>
 <title>Arrow Function Demo</title>
</head>
<body>
<h2>Arrow Function Examples</h2>
<div id="result"></div>

<script>
 let output = "";

 // Implicit return
 const double = n => n * 2;
 output += "Double of 4: " + double(4) + "<br>";

 // Return object
 const getUser = () => ({ name: "Ravi", age: 30 });
 const user = getUser();
 output += `User Name: ${user.name}, Age: ${user.age}<br>`;

 // 'this' inside arrow
```

```
  const person = {
    name: "Meena",
    show: function() {
      const arrow = () => "Name is " + this.name;
      output += arrow() + "<br>";
    }
  };
  person.show();
  document.getElementById("result").innerHTML = output;
</script>
</body>
</html>
```

Output on screen:

Double of 4: 8

User Name: Ravi, Age: 30

Name is Meena

34.4 When to Use Arrow Functions

✓ Callback functions (e.g. in map, filter, forEach)
✓ Short utility functions
Avoid for object methods or constructors

34.5 Quiz Time

Q1. What is the output?

```
const add = (a, b) => a + b;

console.log(add(2, 3));
```

Answer: 5

Q2. Which of the following is invalid?

A) const greet = name => "Hi " + name;
B) const greet = (name) => { return "Hi " + name; };
C) const greet = name => { "Hi " + name };

Correct Answer: C
(No return in block body)

Q3. What happens here?

```
const person = {
  name: "Alex",
  greet: () => console.log("Hi " + this.name)
};person.greet();
```

Answer: Hi undefined
(Arrow function does not have its own this)

35. JavaScript RegExp (Regular Expressions)

A Regular Expression is a sequence of characters that forms a search pattern. Used for string matching, replacing, validating, and splitting strings.

RegExp Syntax:-

```
// Literal
let regex1 = /pattern/;
// Constructor
let regex2 = new RegExp("pattern");
```

Modifiers (flags):

- **i –** Case-insensitive
- **g –** Global search
- **m –** Multi-line search

35.1 Common RegExp Methods

Method	Description	Used With
test()	Returns true/false if match exists	RegExp
exec()	Returns details of first match or null	RegExp
match()	Returns array of matches	String
matchAll()	Returns iterator of all matches	String
replace()	Replaces match with new string	String
search()	Returns index of match or -1	String
split()	Splits string based on pattern	String

35.1.1 Practical Example: test() and exec()

```
<!DOCTYPE html>
<html>
<body>
<h2>RegExp test() and exec()</h2>
<div id="result"></div>
<script>
  const text = "Welcome to JavaScript!";
  const regex = /javascript/i;

  const isMatch = regex.test(text); // true
  const details = regex.exec(text); // ["JavaScript"]

  document.getElementById("result").innerHTML =
    `Match found: ${isMatch}<br>Matched value: ${details[0]}`;
</script>
</body>
</html>
```

Output:

Match found: true

Matched value: JavaScript

35.2 Common RegExp Patterns

Pattern	Description	Example	Matches
.	Any character	/h.t/	hat, hot, hit
^	Start of string	/^Hi/	Hi there
$	End of string	/end$/	it's the end
*	0 or more occurrences	/go*gle/	ggle, gogle
+	1 or more	/go+gle/	gogle, gooogle
\d	Any digit	/\d/	1, 23, 5
\w	Word characters	/\w+/	hello, hi123
\s	Space	/\s/	Matches space/tab
[]	Set of characters	/[aeiou]/	Vowels
`	`	OR operator	`/yes

Practical 35.1.2 : Search and Replace with replace()

```
<!DOCTYPE html>
<html>
<body>
<h2>Replace Word Using RegExp</h2>
<p id="demo">JavaScript is awesome!</p>
<button onclick="replaceWord()">Replace</button>

<script>
function replaceWord() {
  let str = document.getElementById("demo").innerHTML;
  let newStr = str.replace(/awesome/i, "<b>powerful</b>");
  document.getElementById("demo").innerHTML = newStr;
}
</script>
</body>
</html>
```

35.3 Real-Life Use Case: Email Validation

```
function isValidEmail(email) {
  const regex = /^[\w.-]+@[a-zA-Z0-9.-]+\.[a-z]{2,6}$/;
  return regex.test(email);
}
console.log(isValidEmail("user@gmail.com")); // true
console.log(isValidEmail("user@gmail"));     // false
```

35.4 Form validation with username,password length, confirm password, email validation

Code:

```
<html>
<head>
<script type="text/javascript" >
function validate()
{
   var username=document.forms["form"]["username"].value
   var pwd=document.forms["form"]["password"].value
   var email=document.forms["form"]["email"].value
   var contact=document.forms["form"]["contact"].value

   if(username=="" || pwd=="" || email=="" || contact=="")
   {
      document.getElementById("error").innerHTML="Required
field must be filled out"
      return false;
   }
}

function matchpassword()
{
   var pwd=document.getElementById("pwd").value
   var cpwd=document.getElementById("cpwd").value
   if(pwd!=cpwd)
   {
      document.getElementById("error").innerHTML="Password
should be same";
```

```
      document.getElementById("pwd").style.borderColor="red"
;
      document.getElementById("cpwd").style.borderColor="red
";
      document.getElementById("pwd").value="";
      document.getElementById("cpwd").value="";
      document.getElementById("pwd").focus();
      return false;
   }
   else
   {
   document.getElementById("error").innerHTML="";
   return true;
   }
}

function passwordValidate()
{
   var pwd=document.getElementById("pwd").value
   if(pwd.length<6)
   {
      document.getElementById("error").innerHTML="Min 6
characters required";
      document.getElementById("pwd").focus();
      return false;
   }
   else if(pwd.length>=6 && pwd.length<10)
   {
         document.getElementById("error").innerHTML="Weak
Password";
         document.getElementById("pwd").focus();
         return false;
   }
   else if(pwd.length>=10 && pwd.length<12)
   {
      document.getElementById("error").innerHTML="Good
Password";
      document.getElementById("pwd").focus();
```

```
      return true;
   }
   else if(pwd.length>=12 && pwd.length<=15)
   {
      document.getElementById("error").innerHTML="Strong
Password";
      document.getElementById("pwd").focus();
      return true;
   }

}

function onlyNumeric(e)
{
   var code=e.keyCode?e.keyCode:e.charCode
   //alert(code)
   if(code<48 || code>57)
   {
      return false;
   }
}

function validateUsername()
{
   var username=document.getElementById("username").value
   var legalChar=/^[A-Z][A-Za-z0-9_]+$/
   if (!legalChar.test(username))
   {
      document.getElementById("username").borderColor="red"
      document.getElementById("username").value=""
      document.getElementById("username").focus()
      document.getElementById("error").innerHTML="Invalid
Username";
            return false;

   }

}
```

```
function validateEmail()
{
   var email=document.getElementById("email").value
   var legalChar=/^[A-Za-z][A-Za-z0-
9_]+[@gmail.com||@yahoo.com||@rediff.com||@yahoo.co.in
]+$/
   if (!legalChar.test(email))
   {
      document.getElementById("email").borderColor="red"
      document.getElementById("email").value=""
      document.getElementById("email").focus()
      document.getElementById("error").innerHTML="Invalid
//Email"
      return false
   }
}
</script>
</head>
<body bgcolor="yellow">
   <div id="error" style="color:red"></div>
      <div>
         <form name="form" onsubmit="return validate()">
               Username*<input type="text" name="username"
id="username" onblur="validateUsername()"><br/><br/>
               Password*<input type="text" name="password"
id="pwd" maxLength="15"
onblur="passwordValidate()"><br><br/>
               Confirm Password*<input type="text"
name="confirmpassword" id="cpwd" maxLength="15"
onblur="return matchpassword()"><br/><br/>
               Email Id*<input type="text" name="email" id="email"
onblur="return validateEmail()"><br/><br/>
               Contact No*<input type="text" name="contact"
onkeypress="return onlyNumeric(event)"><br/><br/>
               <button type="submit">Submit</button>
         </form>
      </div>
</body> </html>
```

36. JavaScript Scope

Scope determines where variables can be accessed. JavaScript has:

1. **Global Scope**
2. **Function Scope**
3. **Block Scope (let, const)**

36.1 Global Scope:- **Variables declared outside functions are globally accessible.**

```
let siteName = "OpenAI";


function printName() {
  console.log(siteName);
}
printName(); // OpenAI
```

36.2 Function Scope:- **Variables declared inside a function using var are only accessible within that function.**

```
function showAge() {
  var age = 25;
  console.log(age);
}
showAge();
// console.log(age); ReferenceError
```

36.3 Block Scope (let, const)

```
{
  let a = 10;
  const b = 20;
}
// console.log(a); ReferenceError
```

Practical

```
<!DOCTYPE html>
<html>
<body>
<h3>Function vs Block Scope</h3>
<div id="scopeOutput"></div>

<script>
var message = "Global Message";
function testScope() {
  var message = "Function Message";
  if (true) {
    let message = "Block Message";
    document.getElementById("scopeOutput").innerHTML +=
      "Inside block: " + message + "<br>";
  }
  document.getElementById("scopeOutput").innerHTML +=
    "Inside function: " + message + "<br>";
}
testScope();
document.getElementById("scopeOutput").innerHTML +=
  "Global scope: " + message;
</script>
</body></html>
```

Output:

Inside block: Block Message

Inside function: Function Message

Global scope: Global Message

36.4 Scope Chain

JavaScript searches from inner → outer until it finds the variable.

```
let user = "Admin";

function outer() {

  function inner() {

    console.log(user); // Admin

  }

  inner();

}

outer();
```

36.5 Real-World Use Case: Avoiding Global Pollution

```
(function () {
  let privateVar = "secret";
  console.log("Inside IIFE:", privateVar);
})();

// console.log(privateVar); // Not accessible
```

36.6 Quiz Questions

Q.1 : What will be the output?

```
let name = "John";
function greet() {
  let name = "Alice";
  console.log(name);
}
greet();
console.log(name);
```

Output: Alice

John

Q.2: What is the result?

```
let regex = /[a-z]{3}/;
console.log(regex.test("Hello"));
```

Output: true

Q.3 Code Problem

Problem:

Write a function that replaces all 4-letter words in a sentence with "****".

Expected Output:

censorFour("This test line will hide four word.")

// "This **** line will hide **** word."

Solution:

```
function censorFour(sentence) {
  return sentence.replace(/\b\w{4}\b/g, "****");
}
console.log(censorFour("This test line will hide four word."));
```

Output:

This ** line will hide **** word.**

37) JavaScript this Keyword

The this keyword refers to **the object it belongs to**, but its value **depends on how a function is called**.

Context	this Refers To
Global	window object (in browsers)
Object Method	The object
Alone (non-strict mode)	Global (window)
Alone (strict mode)	undefined
Function (non-strict)	Global (window)
Function (strict)	undefined
Arrow Function	Lexical (outer) this
In Event Handler	The element triggering the event

37.1 Global Scope

```
<!DOCTYPE html>
<html>
<body>
<h3>Global `this`</h3>
<p id="globalResult"></p>

<script>
document.getElementById("globalResult").innerHTML =
 (this === window) ? "Yes, 'this' is window" : "No, 'this' is not
window";
</script>
</body>
</html>
```

Output: Yes, 'this' is window

37.2 Inside an Object Method

```
<!DOCTYPE html>
<html>
<body>
<h3>Object Method</h3>
<p id="objResult"></p>

<script>
const person = {
 name: "Alice",
 greet: function () {
  return "Hello " + this.name;
 }
};
document.getElementById("objResult").innerHTML = person.greet();
</script>
</body>
</html>
```

Output: Hello Alice

37.3 this in a Function (non-strict)

```
function show() {
  console.log(this); // refers to window
}
show();
```

37.4 this in a Function (strict mode)

```
"use strict";
function show() {
  console.log(this); // undefined
}
show();
```

37.5 Pitfall: this in Nested Functions

```
const user = {
  name: "Tom",
  greet: function () {
    function inner() {
      console.log(this.name); // undefined (this refers to
global)
    }
    inner();
  }
};
user.greet();
```

37.6 Fix with Arrow Function:

```
const user = {
  name: "Tom",
  greet: function () {
    const inner = () => {
      console.log(this.name); // 'Tom' (lexical binding)
    };
    inner();
  } };
user.greet();
```

37.7 Arrow Functions do NOT have their own this

```
const person = {
  name: "John",
  greet: () => {
    console.log(this.name); // undefined (refers to global,
not person)
  }
};
person.greet();
```

37.8 Use regular function to access correct this:

```
const person = {
  name: "John",
  greet: function () {
    console.log(this.name); // John
  }
};
person.greet();
```

37.9 this in Event Handlers

```
<!DOCTYPE html>
<html>
<body>
<button onclick="showThis(this)">Click Me</button>
<p id="eventThis"></p>

<script>
function showThis(element) {
  element.innerText = "Clicked!";
  document.getElementById("eventThis").innerHTML =
    "this.innerText = " + element.innerText;
}
</script>
</body>
</html>
```

Output on Click:

Clicked!

this.innerText = Clicked!

37.10 Real-World Use Case: Using this in a Class

```
class Car {
  constructor(brand) {
    this.brand = brand;
  }
  showBrand() {
    return `Car brand is ${this.brand}`;
  }
}

const myCar = new Car("Tesla");
console.log(myCar.showBrand());
```

Output:

Car brand is Tesla

37.11 Quiz Questions

Q1: What is the value of this inside an arrow function declared in the global scope?

A) undefined

B) The object

C) The global object

D) Depends on call site

Q2: What's the output?

```
const obj = {
  name: "React",
  getName: () => console.log(this.name)
```

```
};
obj.getName();
```

Output:

undefined

Q.3 Code Problem

Write a function Person with a method introduce that logs:
"Hi, I am [name]", using this. Then call the method.

Expected Output:

Hi, I am Alice

Solution:

```
function Person(name) {
  this.name = name;
  this.introduce = function () {
    console.log("Hi, I am " + this.name);
  };
}

const p1 = new Person("Alice");
p1.introduce();
```

Output:

Hi, I am Alice

38) JavaScript call(), apply(), and bind()

JavaScript functions are **first-class citizens**, meaning they can be treated like any other object: assigned to variables, passed as arguments, and returned from other functions. One powerful feature stemming from this is the ability to dynamically set the context (this) in which a function is executed.

JavaScript provides three methods—**call()**, **apply()**, and **bind()**—to do just that. These methods come from the Function prototype and are available to every function in JavaScript.

Feature	call()	apply()	bind()
Executes Now?	Yes	Yes	No
Args Format	Comma-separated	Array	Comma-separated
Returns	Result	Result	New function
Typical Use	Method borrowing	Argument arrays	Delayed callbacks

What is this in JavaScript?

this keyword refers to the object that is executing the current function. Its value depends on how the function is called, not where it's defined.

- **In global scope or regular function call**: this refers to the global object (or undefined in strict mode).
- **Inside a method**: this refers to the object owning the method.
- **In arrow functions**: this is lexically bound (inherits from parent scope).
- **With call, apply, or bind**: this is explicitly set.

38.1 call() – Invoke Function with Explicit this and Individual Arguments

call() is a method used to **invoke a function immediately**, with a specified this value and **arguments passed individually**.

Syntax:

```
func.call(thisArg, arg1, arg2, ...)
```

Explanation:

- thisArg is the object you want to use as this.
- You can pass arguments one by one.

Use Case: Method borrowing, dynamic method invocation.

Example:

```
<!DOCTYPE html>
<html>
<body>
<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + " from " + city + ", " +
country;
  }
};

const user = {
  firstName: "John",
  lastName: "Doe"
};
const result = person.fullName.call(user, "New York", "USA");
document.write(result);  // John Doe from New York, USA
</script>
</body>
</html>
```

38.2 Apply()

apply() is almost identical to call(), but it accepts arguments as an array or array-like object.

Syntax:

```
func.apply(thisArg, [argsArray])
```

Use Case: When arguments are already in an array (e.g., when working with arguments, Math.max, or spread operations).

Example:

```
<!DOCTYPE html><html><body>
<script>
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + " from " + city + ", " +
country;
  }
};
const user = {
  firstName: "Alice",
  lastName: "Smith"
};
const result = person.fullName.apply(user, ["London", "UK"]);
document.write(result);  // Alice Smith from London, UK
</script>
</body>
</html>
```

38.3 Bind()

bind() does not execute the function. Instead, it returns a **new function** with a permanently bound this and optional preset arguments.

Syntax:

```
let boundFunc = func.bind(thisArg, arg1, arg2, ...)
```

Use Case: Useful in event handlers, callbacks, and preserving this in asynchronous code.

Example:

```
<!DOCTYPE html>
<html>
<body>
<script>
const person = {
  firstName: "Emma",
  lastName: "Watson",
  fullName: function() {
```

```
    return this.firstName + " " + this.lastName;
  }
};

const print = person.fullName;
document.write(print()); // undefined undefined
const boundPrint = person.fullName.bind(person);
document.write("<br>" + boundPrint()); // Emma Watson
</script>
</body>
</html>
```

38.4 Quiz Question

Q.1: What's the output of the following?

```
const person = { name: 'Eva' };

function printName(age) {

  return this.name + ' is ' + age + ' years old.';

}

const boundFunc = printName.bind(person, 28);

console.log(boundFunc());
```

Output: Eva is 28 years old.

Q.2 Coding Challenge Problem Statement:

Create a function that multiplies two numbers. Bind it to an object and pass default value for the first number.

Answer:-

```
const multiplier = {

  multiply: function (a, b) {

    return a * b;

  } };

const double = multiplier.multiply.bind(multiplier, 2);

console.log(double(5));   //Output: 10
```

39. JavaScript Sets

A **Set** is a collection of **unique values**. It is one of the built-in objects in JavaScript introduced in **ES6** (ECMAScript 2015) to store **unordered** values without duplicates.

Unlike arrays, **Sets do not allow duplicate entries**, and each value can occur only once.

Key Features of Sets

- Automatically removes duplicates
- Maintains insertion order
- Can hold **any data type**: strings, numbers, objects, etc.
- Values are unique by **strict equality** (===)
- Not index-based (unlike arrays)
- Has built-in **utility methods**

Syntax:-

```
const mySet = new Set();              // Creates an empty set
const mySet2 = new Set([1, 2, 3, 4]);   // Set initialized with values
```

Method / Property	Description
add(value)	Adds a new value if not already present
delete(value)	Deletes a specified value
has(value)	Returns true if value exists, otherwise false
clear()	Removes all values from the set
size (property)	Returns the number of elements in the set
forEach(callback)	Iterates over all values in the order they were inserted
values() / keys()	Returns an iterator object of all values in insertion order
entries()	Returns [value, value] for compatibility with Map

39.1 Sets do not allow duplicates:

```
const s = new Set();
s.add(5);
s.add(5); // Ignored
console.log(s.size); // 1
```

39.2 Objects are unique by reference:

```
const s = new Set();
s.add({a: 1});
s.add({a: 1}); // Considered different objects
console.log(s.size); // 2
```

39.3 Convert Between Array and Set

```
// Array to Set (remove duplicates)
const nums = [1, 2, 2, 3];
const uniqueNums = [...new Set(nums)];
console.log(uniqueNums); // [1, 2, 3]


// Set to Array
const mySet = new Set(['x', 'y']);
const arr = Array.from(mySet);
console.log(arr); // ['x', 'y']
```

Practical Code Example

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Sets</title></head>
<body>

  <h2>JavaScript Sets Example</h2>

<div id="result"></div>
```

```
 <script>
   const output = [];

   // Create and add values
   const fruitSet = new Set();
   fruitSet.add("apple");
   fruitSet.add("banana");
   fruitSet.add("apple"); // duplicate
   fruitSet.add(100);
   fruitSet.add({type: "fruit"});
   output.push("Set Size: " + fruitSet.size); // 4
   output.push("Has 'banana'? " + fruitSet.has("banana")); // true

   // Delete a value
   fruitSet.delete("banana");
   output.push("After deleting 'banana': " + fruitSet.has("banana"));
// false

   // Iterate through the set
   fruitSet.forEach(value => {
     output.push("Value: " + JSON.stringify(value));
   });

   document.getElementById("result").innerHTML =
output.join("<br>");
  </script>
</body>
</html>
```

Output in Browser

Set Size: 4

Has 'banana'? true

After deleting 'banana': false

Value: "apple"

Value: 100

Value: {"type":"fruit"}

39.4 Tricky Question

Q.1 : What's the output?

```
const s = new Set();
s.add("1");
s.add(1);
console.log(s.size);
```

Answer: 2 [… Because "1" (string) and 1 (number) are **different types**.]

Q.2 Mini Exercise Problem Statement:

Write a function that returns the **common elements** between two arrays using a Set.

Ans:-

```
<!DOCTYPE html><html>
<head><title>Set Intersection</title></head>
<body> <div id="commonResult"></div>
 <script>
  function getCommon(arr1, arr2) {
   const set1 = new Set(arr1);
   return arr2.filter(item => set1.has(item));
  }
  const array1 = [1, 2, 3, 4, 5];
  const array2 = [4, 5, 6, 7];
  const result = getCommon(array1, array2);
document.getElementById("commonResult").innerHTML =
"Common Elements: " + result;
 </script>
</body></html>
```

Output:

Common Elements: 4,5

40) JavaScript Maps

A **Map** is a built-in JavaScript object that allows you to store **key-value pairs**, where **keys can be of any data type** — including **objects, functions, or primitives**.

It was introduced in **ES6 (ECMAScript 2015)** and provides a more flexible alternative to plain JavaScript objects when you need **ordered, iterable, and dynamically typed keys**.

Why Use a Map Instead of an Object?

Feature	Map	Object
Key types	Any (primitive or object)	Only string or symbol
Maintains insertion order	Yes	Not guaranteed
Iterable	Yes (forEach, for...of)	Not directly iterable
Key count	map.size	Object.keys(obj).length
Performance for frequent add/remove	Better	Slower with lots of keys

Creating a Map:-

```
const map = new Map();
const map2 = new Map([
  ['name', 'John'],
  ['age', 25]
]);
```

40.1 Common Map Methods

Method	Description
set(key, value)	Adds or updates a key-value pair
get(key)	Returns the value associated with the key
has(key)	Returns true if the key exists

Method	Description
delete(key)	Deletes the specified key-value pair
clear()	Removes all key-value pairs
size (property)	Returns the number of entries in the map
keys()	Returns an iterator of keys
values()	Returns an iterator of values
entries()	Returns an iterator of [key, value] pairs
forEach()	Executes a callback for each key-value pair

Practical :-

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Map Example</title></head>
<body>
  <h2>JavaScript Map Example</h2>
  <div id="mapResult"></div>
  <script>
    const myMap = new Map();
    // Add key-value pairs
    myMap.set("name", "Alice");
    myMap.set("age", 30);
    myMap.set("country", "USA");

    // Use object as key
    const user = { id: 1 };
    myMap.set(user, "admin");

    let result = `Size of Map: ${myMap.size}<br>`;
    result += `Name: ${myMap.get("name")}<br>`;
    result += `Has 'age': ${myMap.has("age")}<br>`;

    // Loop over map
    myMap.forEach((value, key) => {
```

```
    result += `${JSON.stringify(key)} => ${value}<br>`;
  });

  document.getElementById("mapResult").innerHTML = result;
 </script>
</body>
</html>
```

Output in Browser:- Size of Map: 4

Name: Alice

Has 'age': true

"name" => Alice

"age" => 30

"country" => USA

{"id":1} => admin

40.2 Important Differences vs Objects

Feature	Map	Object
Key data type	Any type (object, number)	Only strings/symbols
Key order	Preserved	Not guaranteed
Iteration	Easy with forEach	Need Object.keys() etc
Size	map.size	Object.keys().length
Performance	Faster for add/delete	Slower for dynamic keys

40.3 Convert Between Map and Object

➤ **Map → Object (only if keys are strings)**

```
const myMap = new Map([
  ['x', 10], ['y', 20]
]);
const obj = Object.fromEntries(myMap);
```

```
console.log(obj); // { x: 10, y: 20 }
```

➤ **Object → Map**

```
const obj = { x: 1, y: 2 };
const myMap = new Map(Object.entries(obj));
console.log(myMap); // Map(2) { 'x' => 1, 'y' => 2 }
```

40.4 Map with Object Keys

```
const map = new Map();
const objKey = { id: 101 };
map.set(objKey, "User 101");
console.log(map.get(objKey)); // "User 101"
```

40.5 Real-Life Use Cases for Maps

Use Case	Benefit
Caching results	Object keys would fail due to collisions
Tracking DOM nodes or elements	Use object references as keys
Frequency counter with non-string keys	Map handles any data type efficiently
Multi-user session data	Object as key for user-specific info

40.6 Looping Through a Map

```
const scores = new Map([
  ["Alice", 90],
  ["Bob", 85],
  ["Charlie", 88]
]);
// for...of
for (let [key, value] of scores) {
  console.log(`${key} => ${value}`);
}
```

Q.1 Challenge: Frequency Counter

```
function frequencyCounter(arr) {
  const freqMap = new Map();
  for (let item of arr) {
    freqMap.set(item, (freqMap.get(item) || 0) + 1);
  }
  return freqMap;
}

const result = frequencyCounter(['a', 'b', 'a', 'c', 'b', 'a']);
for (let [key, value] of result) {
  console.log(`${key}: ${value}`);
}
```

Output:

a: 3

b: 2

c: 1

41) JavaScript Object: Definition & Properties

An **object** in JavaScript is a complex data type that allows you to store collections of **key-value pairs**. Each key is a **property**, and its value can be any data type: primitive, array, function, or even another object. Objects are used to **group related data and functions** together in a structured way. They provide a logical model of real-world entities.

Objects = Foundation of JavaScript programming.

How to Create an Object

```
const person = {
  name: "Alice",
  age: 25
};
```

Using new Object()

```
const person = new Object();
person.name = "Alice";
person.age = 25;
```

Using a Constructor Function

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const person1 = new Person("Alice", 25);
```

41.1 Object Properties

A **property** is a key-value pair associated with an object.

```
const user = {
  name: "John",
  age: 30,
  isAdmin: true
};
```

Accessing Properties

```
console.log(user.name);      // Dot notation → John
console.log(user["age"]);    // Bracket notation → 30
```

Adding New Properties

```
user.email = "john@example.com";
```

Deleting Properties

```
delete user.isAdmin;
```

Practical Example:

```
<!DOCTYPE html>
<html>
<head><title>JavaScript Object</title></head>
<body>
  <h2>JavaScript Object Properties</h2>
  <div id="output"></div>

  <script>
    const book = {
      title: "JavaScript Basics",
      author: "Alex",
      pages: 200,
      isPublished: true,
      details: function() {
        return `${this.title} by ${this.author}, ${this.pages} pages.`;
      }
    };

    // Modify property
    book.pages = 250;

    // Add new property
    book.publisher = "Tech Books";

    // Output
    let output = book.details() + "<br>";
    output += "Publisher: " + book.publisher + "<br>";
    output += "Is Published: " + book["isPublished"];

    document.getElementById("output").innerHTML = output;
  </script>
</body>
</html>
```

Output:

JavaScript Basics by Alex, 250 pages.

Publisher: Tech Books

Is Published: true

41.2 Property Attributes

Each property has hidden attributes:

Attribute	Description
value	The value of the property
writable	Can it be changed?
enumerable	Can it be looped over?
configurable	Can it be deleted or changed?

Use Object.defineProperty() to control attributes:

```
Object.defineProperty(user, 'role', {
  value: 'admin',
  writable: false,
  enumerable: true
});
```

Looping Through Object Properties

```
const person = {
  name: "Sam",
  age: 40,
  city: "Delhi"    };
for (let key in person) {
  console.log(`${key}: ${person[key]}`);
}
```

Output:

name: Sam

age: 40

city: Delhi

41.3 Difference Between Dot & Bracket Notation

Dot Notation	Bracket Notation
obj.key	obj["key"]
Simpler syntax	Required for dynamic keys
Cannot use spaces	Can use strings with spaces

42) JavaScript Object Methods

A method in JavaScript is a function that is a property of an object.

Methods are object functions that can operate on object data (i.e., its properties).

Why Use Object Methods?

- To group logic with data.
- To perform actions on object properties.
- To create reusable behavior for similar objects.

Basic Syntax

```
const person = {
  name: "Alice",
  greet: function() {
    return "Hello, " + this.name;
  }
};
```

Here, greet is a **method** of the person object.

42.1 Using the this Keyword

Inside an object method, this refers to the object itself.

```
const user = {
  username: "sam123",
  showUsername: function() {
    return this.username;
  }
};
console.log(user.showUsername()); // Output: sam123
```

42.2 Defining Methods in Different Ways

1. Traditional Method Syntax

```
const obj = {
  method: function() {
    return "Hello!";
  }
};
```

2. Shorthand Method Syntax (ES6)

```
const obj = {
  method() {
    return "Hello!";
  }
};
```

Example: Object with Multiple Methods

```
<!DOCTYPE html>
<html>
<head>
  <title>Object Methods</title>
</head>
```

```
<body>
  <h2>Object Methods Example</h2>
  <div id="output"></div>

  <script>
    const student = {
      firstName: "Riya",
      lastName: "Sharma",
      marks: [80, 90, 75],
      fullName() {
        return this.firstName + " " + this.lastName;
      },
      average() {
        const sum = this.marks.reduce((a, b) => a + b, 0);
        return (sum / this.marks.length).toFixed(2);
      }
    };
    let output = "Full Name: " + student.fullName() + "<br>";
    output += "Average Marks: " + student.average();
    document.getElementById("output").innerHTML = output;
  </script>
</body>
</html>
```

Output:

Full Name: Riya Sharma

Average Marks: 81.67

42.3 Use Case: Employee Salary Calculator

```
const employee = {
  name: "Raj",
  baseSalary: 30000,
  bonus: 5000,
  totalSalary: function() {
    return this.baseSalary + this.bonus;
```

```
  }
};
console.log("Total Salary of", employee.name, "is",employee.totalSalary());
```

Output: Total Salary of Raj is 35000

42.4 Object Methods with Parameters

```
const calculator = {
  add(x, y) {
    return x + y;
  },
  multiply(x, y) {
    return x * y;
  }
};
console.log(calculator.add(5, 3));      // 8
console.log(calculator.multiply(5, 3)); // 15
```

42.5 Built-In Object Methods

Every object inherits built-in methods from Object.prototype, like:

Method	Description
Object.keys(obj)	Returns an array of property names
Object.values(obj)	Returns an array of property values
Object.entries(obj)	Returns key-value pairs as nested arrays
Object.assign()	Copies properties to another object
Object.hasOwnProperty()	Checks if property exists

42.6 Common Interview Questions

Q.1 What's the difference between function and method?

Function	Method
Independent	Part of an object
Called directly	Called on an object

Q.2 What does this refer to in an object method?

Answer: this refers to the object the method belongs to.

Q.3 Mini Coding Challenge:- Problem

Create a product object that includes:

- name
- price
- tax (in %)
- method finalPrice() that returns total price after tax.

Expected Output:

Product: Laptop

Final Price: ₹55000

Solution:

```
<!DOCTYPE html>
<html>
<body>
 <h2>Product Price Calculator</h2>
 <div id="result"></div>
 <script>
  const product = {
   name: "Laptop",
   price: 50000,
   tax: 10, // in percent
   finalPrice() {
    return this.price + (this.price * this.tax / 100);
   }
  };
```

```
  document.getElementById("result").innerHTML =
    "Product: " + product.name + "<br>" +
    "Final Price: ₹" + product.finalPrice();
  </script>
</body>
</html>
```

43) JavaScript Object Display

Object Display refers to various ways you can **output or visualize the content of an object** in JavaScript.

JavaScript objects are complex data types. When you display them, the way they appear depends on *how* you try to access or print them.

Why Is Object Display Important?

- Helps debug object data.
- Lets you inspect key-value pairs.
- Useful in rendering dynamic content in web apps.

Common Ways to Display an Object

Technique	Description
console.log(obj)	Debugging purpose in console
JSON.stringify(obj)	Converts object to a readable string
Loop (for...in)	Access each property manually
Object.entries(obj)	Returns an array of key-value pairs
Display in HTML (DOM)	Write object properties inside the webpage

43.1. Display Using console.log()

```
const person = { name: "Amit", age: 25 };
console.log(person);
```

Output (Console): {name: "Amit", age: 25}

43.2. Display Using JSON.stringify()

```
const person = { name: "Amit", age: 25 };
console.log(JSON.stringify(person));
```

Output: {"name":"Amit","age":25}

You can also format it:

```
console.log(JSON.stringify(person, null, 2));
```

Formatted Output:

```
{
  "name": "Amit",
  "age": 25
}
```

43.3. Display Using for...in Loop

```
const person = { name: "Amit", age: 25 };
for (let key in person) {
  console.log(key + ": " + person[key]);
}
```

Output:

```
name: Amit
age: 25
```

43.4. Display Using Object.entries()

```
const person = { name: "Amit", age: 25 };
Object.entries(person).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);        });
```

Output:

name: Amit

age: 25

43.5. Example :- Display in HTML

```
<!DOCTYPE html>
<html>
<head><title>Display Object</title></head>
<body>
<h2>Object Display Example</h2>
<div id="info"></div>

<script>
  const employee = {
    id: 101,
    name: "Neha",
    department: "IT",
    salary: 35000
  };

  let output = "<ul>";
  for (let key in employee) {
    output += `<li><strong>${key}</strong>: ${employee[key]}</li>`;
  }
  output += "</ul>";
  document.getElementById("info").innerHTML = output;
</script>

</body>
</html>
```

Output:-

- id: 101
- name: Neha
- department: IT
- salary: 35000

43.6. Use alert() to Display Object Properties

```
const car = { brand: "Honda", model: "City" };

alert(`Brand: ${car.brand}\nModel: ${car.model}`);
```

43.7. Create Dynamic HTML Table from Object

```
<!DOCTYPE html>
<html>
<head><title>Table Display</title></head>
<body>
<h2>Employee Details</h2>
<table border="1" id="empTable"></table>
<script>
  const employee = {
    Name: "Ravi",
    Designation: "Developer",
    Age: 28,
    Experience: "4 Years"
  };
  let tableHTML = "<tr><th>Key</th><th>Value</th></tr>";

  for (let key in employee) {
    tableHTML +=
`<tr><td>${key}</td><td>${employee[key]}</td></tr>`;
  }

  document.getElementById("empTable").innerHTML = tableHTML;
</script>

</body>
</html>
```

Output:

Key	Value
Name	Ravi
Designation	Developer
Age	28
Experience	4 Years

44) JavaScript Object Accessors

In JavaScript, **Object Accessors** are special methods used to **get** or **set** the value of an object property. Instead of accessing properties directly, you can define **getter** and **setter** functions to control access and add custom behavior when reading or updating a property.

Why Use Accessors?

- Encapsulation: Hide the internal representation of the property.
- Validation: Validate data before setting a value.
- Computed Properties: Return dynamic values when getting a property.
- Side Effects: Trigger actions when properties are read or modified.

44.1 Two Types of Accessors

Accessor	Purpose	Syntax Example
Getter	To access (read) a property value	get propertyName() { ... }
Setter	To modify (write) a property value	set propertyName(value) { ... }

44.1.1 Basic Getter Example

```
const person = {
  firstName: "Anita",
  lastName: "Sharma",
  get fullName() {
    return `${this.firstName} ${this.lastName}`;
  }
};
console.log(person.fullName);
```

Output:

Anita Sharma

Explanation:

- fullName is not stored as a property but computed dynamically by combining firstName and lastName.

44.1.2. Basic Setter Example

```
const person = {
  firstName: "Anita",
  lastName: "Sharma",
  set fullName(name) {
    const parts = name.split(" ");
    this.firstName = parts[0];
    this.lastName = parts[1];
  }
};
person.fullName = "Ravi Kumar";
console.log(person.firstName);  // Ravi
console.log(person.lastName);   // Kumar
```

Output:

Ravi

Kumar

Explanation:

- **The setter fullName splits a full name string and updates firstName and lastName properties accordingly.**

44.1.3. Getter and Setter Together

```
const employee = {
  _salary: 30000,
  get salary() {
    return this._salary;
  },
```

```
set salary(amount) {
   if (amount > 0) {
     this._salary = amount;
   } else {
     console.log("Invalid salary amount");
   }
 }
};
console.log(employee.salary); // 30000
employee.salary = 40000;
console.log(employee.salary); // 40000
employee.salary = -100;      // Invalid salary amount
console.log(employee.salary); // 40000 (unchanged)
```

44.1.4. Use Case: Validate Input Before Setting

```
const user = {
 _age: 25,
 get age() {
   return this._age;
 },
 set age(value) {
   if (value < 0) {
     console.log("Age cannot be negative.");
   } else {
     this._age = value;
   } } };
user.age = 30;
console.log(user.age);  // 30
user.age = -5;        // Age cannot be negative.
console.log(user.age);  // 30 (unchanged)
```

44.1.5. Object Accessors in Classes

```
class Product {
  constructor(name, price) {
    this.name = name;
    this._price = price;
  }
  get price() {
    return this._price;
  }
  set price(value) {
    if (value > 0) {
      this._price = value;
    } else {
      console.log("Price must be positive");
    }
  }
}
const prod = new Product("Book", 150);
console.log(prod.price);  // 150
prod.price = 200;
console.log(prod.price);  // 200
prod.price = -50;        // Price must be positive
console.log(prod.price);  // 200 (unchanged)
```

How Accessors Work Internally

- The **getter** function executes when you access the property.
- The **setter** function executes when you assign a value to the property.
- You **cannot** have both an accessor and a data property with the same name on the same object.

Practical Example

```
<!DOCTYPE html>
<html>
<head>
 <title>Object Accessors Example</title>
</head>
<body>
 <h2>JavaScript Object Accessors Demo</h2>
 <div id="output"></div>

 <script>
  const person = {
   firstName: "Priya",
   lastName: "Singh",

   get fullName() {
    return `${this.firstName} ${this.lastName}`;
   },

   set fullName(name) {
    const parts = name.split(" ");
    if (parts.length === 2) {
     this.firstName = parts[0];
     this.lastName = parts[1];
    } else {
     alert("Please provide first and last name separated by space");
    }
   }
  };

  let outputDiv = document.getElementById("output");
  outputDiv.innerHTML += `<p>Initial Full Name:
${person.fullName}</p>`;

  person.fullName = "Rahul Verma";
  outputDiv.innerHTML += `<p>After Setter, First Name:
${person.firstName}</p>`;
  outputDiv.innerHTML += `<p>After Setter, Last Name:
${person.lastName}</p>`;
  outputDiv.innerHTML += `<p>Updated Full Name:
${person.fullName}</p>`;
```

```
  // Invalid setter use
  person.fullName = "SingleName";  // Alerts user
 </script>
</body>
</html>
```

Output:-

- Initial Full Name: Priya Singh
- After Setter, First Name: Rahul
- After Setter, Last Name: Verma
- Updated Full Name: Rahul Verma
- (An alert pops up for invalid input: "Please provide first and last name separated by space")

45) JavaScript Object Constructors

In JavaScript, an **Object Constructor** is a special function used to **create multiple instances** of similar objects with the same structure and behavior.

Why Use Object Constructors?

- To create many objects efficiently without repeating code.
- To provide a blueprint for objects (like classes in other languages).
- To initialize object properties when creating new instances.
- To add methods shared by all instances via prototype.

Constructor Function Syntax

```
function Person(firstName, lastName, age) {
  this.firstName = firstName;  // property
  this.lastName = lastName;    // property
  this.age = age;              // property
  this.fullName = function() { // method
    return `${this.firstName} ${this.lastName}`;
  }; }
```

- The keyword this refers to the new object created by the constructor.
- You call it with the new keyword to create an instance.

Creating Objects Using Constructor

```
const person1 = new Person("Anita", "Sharma", 28);
const person2 = new Person("Rahul", "Verma", 32);
console.log(person1.fullName()); // Anita Sharma
console.log(person2.fullName()); // Rahul Verma
```

Output

Anita Sharma

Rahul Verma

45.1 Example: Constructor with Methods on Prototype

To save memory and avoid copying methods for every instance, you can define methods on the constructor's **prototype**:

```
function Employee(name, salary) {
  this.name = name;
  this.salary = salary;
}
Employee.prototype.getDetails = function() {
  return `${this.name} earns $${this.salary}`;
};
const emp1 = new Employee("Riya", 50000);
const emp2 = new Employee("Karan", 60000);
console.log(emp1.getDetails());  // Riya earns $50000
console.log(emp2.getDetails());  // Karan earns $60000
```

Output

Riya earns $50000

Karan earns $60000

45.2 Use Case: Factory of Similar Objects

If you have a system managing many similar entities, constructors let you generate them dynamically with custom data.

Object Literal	Constructor Function
Creates one object	Can create multiple objects using new
Syntax: { name: 'John', age: 30 }	Syntax: new Person('John', 'Doe', 30)
No prototype sharing by default	Can share methods via prototype

45.3 Adding Properties & Methods After Construction

You can add properties or methods anytime:

```
const person = new Person("Anita", "Sharma", 28);

// Add a method later

Person.prototype.greet = function() {

  return `Hello, ${this.firstName}!`;

};

console.log(person.greet()); // Hello, Anita!
```

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Object Constructors Demo</title>
</head>
<body>
  <h2>Employee Details</h2>
  <div id="empOutput"></div>
  <script>
    function Employee(name, department, salary) {
      this.name = name;
      this.department = department;
      this.salary = salary;
    }
```

```
  Employee.prototype.getDetails = function() {
    return `${this.name} works in ${this.department} and earns
$${this.salary}`;
  };
  const emp1 = new Employee("Sita", "HR", 45000);
  const emp2 = new Employee("Raj", "IT", 55000);
  const output = document.getElementById("empOutput");
  output.innerHTML += `<p>${emp1.getDetails()}</p>`;
  output.innerHTML += `<p>${emp2.getDetails()}</p>`;
 </script>
</body>
</html>
```

Output :-

- Sita works in HR and earns $45000
- Raj works in IT and earns $55000

46) JavaScript Object Prototypes

In JavaScript, **every object** has a hidden internal property called [[Prototype]] (commonly accessed via __proto__ or Object.getPrototypeOf()), which points to another object. This linked object is called the **prototype**.

- The prototype contains properties and methods that the original object can **inherit**.
- This mechanism is known as **prototypal inheritance**.
- It allows JavaScript to share behavior (methods and properties) between objects efficiently.

Why Are Prototypes Important?

- To **avoid duplication** of methods in every object instance.
- To enable **inheritance** and **code reuse**.
- To provide a dynamic way to add or override methods that all objects can access.

- Fundamental to understanding how JavaScript objects and inheritance work.

Prototype Chain

When you try to access a property or method on an object, JavaScript:

1. Looks for the property on the object itself.
2. If not found, it looks at the object's prototype.
3. If still not found, looks at the prototype's prototype, and so on...
4. Until it reaches null (end of the chain).

46.1 How to Access an Object's Prototype?

```
const obj = {};
console.log(Object.getPrototypeOf(obj));  // Outputs: Object.prototype
```

Or using __proto__ (not recommended but often used):

```
console.log(obj.__proto__);
```

46.2 Example: Prototype with Constructor Function

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
// Add method to prototype
Person.prototype.greet = function() {
  return `Hello, my name is ${this.name}`;
};
const p1 = new Person("Anita", 25);
const p2 = new Person("Ravi", 30);
console.log(p1.greet());  // Hello, my name is Anita
console.log(p2.greet());  // Hello, my name is Ravi
```

Output

Hello, my name is Anita

Hello, my name is Ravi

Explanation:

- The greet method is not defined on each p1 or p2 instance but inherited from Person.prototype.
- This saves memory because only one greet function exists and is shared.

46.3 Prototypes in Object Literals

```
const obj = {
  a: 10,
  b: 20
};
console.log(obj.__proto__ === Object.prototype);  // true
```

Objects created via literals have Object.prototype as their prototype.

46.4 Adding Properties to Prototype Later

You can add methods/properties to prototype even after objects are created:

```
function Car(model) {
  this.model = model;
}
const car1 = new Car("Tesla");
Car.prototype.getModel = function() {
  return this.model;
};
console.log(car1.getModel());  // Tesla
```

46.5 Prototype Chain Example

```
const grandParent = { grandParentProp: "I am grandparent" };

const parent = Object.create(grandParent);

parent.parentProp = "I am parent";

const child = Object.create(parent);

child.childProp = "I am child";

console.log(child.childProp);      // I am child

console.log(child.parentProp);     // I am parent (inherited)

console.log(child.grandParentProp); // I am grandparent (inherited)
```

Summary of Prototype Chain

Object	**Prototype**
child	parent
parent	grandParent
grandParent	Object.prototype
Object.prototype	null (end of chain)

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Prototypes Demo</title>
</head>
<body>
  <h2>Prototype Chain Demonstration</h2>
  <div id="output"></div>
  <script>
    function Animal(name) {
      this.name = name;
    }
    Animal.prototype.speak = function() {
      return `${this.name} makes a noise.`;
```

```
  };
  function Dog(name, breed) {
    Animal.call(this, name);
    this.breed = breed;
  }

  // Inherit prototype
  Dog.prototype = Object.create(Animal.prototype);
  Dog.prototype.constructor = Dog;

  Dog.prototype.speak = function() {
    return `${this.name} barks.`;
  };

  const dog1 = new Dog("Buddy", "Golden Retriever");
  const dog2 = new Dog("Max", "Beagle");

  const output = document.getElementById("output");
  output.innerHTML += `<p>${dog1.speak()}</p>`;
  output.innerHTML += `<p>${dog2.speak()}</p>`;

  // Prototype chain checks
  output.innerHTML += `<p>dog1.__proto__ === Dog.prototype:
${dog1.__proto__ === Dog.prototype}</p>`;
  output.innerHTML += `<p>Dog.prototype.__proto__ ===
Animal.prototype: ${Dog.prototype.__proto__ ===
Animal.prototype}</p>`;
  </script>
</body>
</html>
```

Output in Browser

- Buddy barks.
- Max barks.
- dog1.**proto** === Dog.prototype: true
- Dog.prototype.**proto** === Animal.prototype: true

47) JavaScript Object Iterables

An **iterable** is any object that implements the **iteration protocol**, meaning it defines how to produce a sequence of values one at a time, which can be looped over using constructs like for...of.

- Examples of built-in iterables: Arrays, Strings, Maps, Sets, TypedArrays, and more.

Why Are Iterables Useful?

- To enable looping over objects using for...of and spread operator (...).
- To allow consumption by APIs expecting iterable input.
- To build custom collections that behave like arrays or sets.

47.1 The Iteration Protocol

An object is iterable if it has a method keyed by Symbol.iterator that returns an **iterator**.

- The iterator is an object with a next() method.
- **Calling next() returns an object with two properties:**

  1.value: The current element.

  2. done: A boolean indicating if iteration is complete.

47.2 Example: Built-in Iterable — Array

```
const arr = [10, 20, 30];
for (const value of arr) {
  console.log(value);
}
```

Output:

10

20

30

47.3 Custom Iterable Example

```
const myIterable = {
 data: [1, 2, 3],
 [Symbol.iterator]() {
  let index = 0;
  const data = this.data;
  return {
   next() {
    if (index < data.length) {
     return { value: data[index++], done:
false };
    } else {
     return { done: true };
    }
   }
  };
 }
};

for (const num of myIterable) {
 console.log(num);
}
```

Output: 1

2

3

Explanation

- The object myIterable defines a method at key Symbol.iterator.
- This method returns an iterator object with a next() method.
- next() returns the next value until the data array is exhausted.

47.4 Using Spread Operator

Iterables can be expanded using spread syntax:

```
const arr = [1, 2, 3];

const arrCopy = [...arr];  // copies elements

console.log(arrCopy);  // [1, 2, 3]
```

Practical Example

```
<!DOCTYPE html>
<html>
<head>
 <title>JavaScript Iterables Demo</title>
</head>
<body>
 <h2>Custom Iterable Example</h2>
 <div id="output"></div>

 <script>
  const myIterable = {
   data: ["apple", "banana", "cherry"],
   [Symbol.iterator]() {
    let index = 0;
    const data = this.data;
    return {
     next() {
      if (index < data.length) {
       return { value: data[index++], done: false };
      } else {
       return { done: true };
      }
     }
    };
   }
  };

  const output = document.getElementById("output");
  let text = '';
  for (const fruit of myIterable) {
   text += `<p>${fruit}</p>`;
  }
  output.innerHTML = text;
 </script>
```

```
</body>
</html>
```

Output:-

- apple
- banana
- cherry

48) JavaScript Object Reference

In JavaScript, **objects are reference types**. This means that when you assign an object to a variable, you're not copying the entire object but only the **reference** (or address) where the object is stored in memory.

- Variables hold a reference pointing to the actual object.
- Multiple variables can reference the same object.
- Changes made via one reference reflect in all references pointing to that object.

Why Object References Matter?

- Understanding references is key to avoiding unintended side effects.
- Helps manage how data is shared or copied.
- Explains why objects behave differently than primitive types when assigned or passed to functions.

Primitive vs Reference Types

Type	Assignment Behavior	Example
Primitive	Copies value	let a = 5; let b = a;
Reference	Copies reference (address)	let obj1 = {x:10}; let obj2 = obj1;

Example: Assigning Objects

```
let obj1 = {name: "John"};
let obj2 = obj1; // obj2 references the same object as obj1
obj2.name = "Jane";
console.log(obj1.name);  // Output: Jane
```

Output:- Jane

Explanation:

- obj2 points to the same object as obj1.
- Changing obj2.name affects obj1.name because both refer to the same object.

Example: Passing Objects to Functions

```
function changeName(person) {
  person.name = "Alice";
}
const personObj = {name: "Bob"};
changeName(personObj);
console.log(personObj.name);  // Output: Alice
```

Explanation:

- personObj is passed by reference.
- The function modifies the original object.

48.1 How to Clone Objects to Avoid Reference Issues

1. Using Object.assign()

```
let original = {a: 1, b: 2};
let copy = Object.assign({}, original);
copy.a = 10;
console.log(original.a);  // 1 (unchanged)
```

2. Using Spread Operator (...)

```
let original = {a: 1, b: 2};

let copy = {...original};

copy.b = 20;

console.log(original.b);  // 2 (unchanged)
```

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Object Reference Demo</title>
</head>
<body>
  <h2>Object Reference and Cloning Demo</h2>
  <pre id="output"></pre>
  <script>
    const output = document.getElementById('output');
    let obj1 = {name: "John", nested: {age: 30}};
    let obj2 = obj1;  // Reference assignment

    obj2.name = "Jane";  // Changes original because same reference
    obj2.nested.age = 40;

    let obj3 = {...obj1};  // Shallow copy using spread operator
    obj3.name = "Alice";
    obj3.nested.age = 50;  // Modifies original nested object (shallow
copy)

    output.textContent = `
    obj1.name: ${obj1.name}      (Expected "Jane")
    obj1.nested.age: ${obj1.nested.age}  (Expected 50 - affected by
shallow copy)

    obj3.name: ${obj3.name}      (Expected "Alice")
    `;
  </script>
</body>
</html>
```

Output:-

obj1.name: Jane (Expected "Jane")

obj1.nested.age: 50 (Expected 50 - affected by shallow copy)

obj3.name: Alice (Expected "Alice")

49) JavaScript Function Definitions

A **function definition** is the syntax used to create a function in JavaScript. Functions are reusable blocks of code designed to perform a specific task.

Why Functions Are Important

- They help organize code into modular, reusable parts.
- They reduce repetition.
- They can accept inputs (parameters) and return outputs.
- Functions enable better maintainability and readability.

49.1 Types of Function Definitions in JavaScript

1. Function Declaration

```
function greet(name) {
  return "Hello " + name;
}
```

- Named function.
- Hoisted — can be called before declaration in the code.

2. Function Expression

```
const greet = function(name) {
  return "Hello " + name;
};
```

- Function assigned to a variable.
- Not hoisted — cannot be called before declaration.

3. Arrow Function (ES6)

```
const greet = (name) => {
  return "Hello " + name;
};
```

- Shorter syntax.
- Does not have its own this.
- Implicit return if only expression.

Examples

```
// Function Declaration
function add(a, b) {
  return a + b;
}

// Function Expression
const multiply = function(a, b) {
  return a * b;
};

// Arrow Function
const subtract = (a, b) => a - b;
```

Output

```
console.log(add(5, 3));      // 8
console.log(multiply(5, 3));  // 15
console.log(subtract(5, 3));  // 2
```

Practical Example:-

```
<!DOCTYPE html><html>
<head>
  <title>JavaScript Function Definitions</title>
</head>
<body>
  <h2>Function Definition Types</h2>
  <pre id="output"></pre>

  <script>
    function add(a, b) {
      return a + b;
    }
const multiply = function(a, b) {
```

```
    return a * b;
  };
  const subtract = (a, b) => a - b;
  const output = document.getElementById('output');
  output.textContent =
    "add(5, 3): " + add(5, 3) + "\n" +
    "multiply(5, 3): " + multiply(5, 3) + "\n" +
    "subtract(5, 3): " + subtract(5, 3) + "\n";
 </script>
</body>
</html>
```

50) JavaScript Function Parameters

Parameters are named variables listed in a function definition. They act as placeholders for values (arguments) passed into the function when called.

Why Are Parameters Important?

- Allow functions to accept input data dynamically.
- Make functions reusable for different inputs.
- Control the behavior of functions depending on arguments.

Basic Syntax

```
function greet(name) {
 return "Hello " + name;
}
greet("John"); // 'John' is the argument passed to parameter `name`
```

50.1 Types of Parameters in JavaScript

1. Regular Parameters:- Defined in the function parentheses, receive passed arguments in order.

```
function sum(a, b) {
 return a + b;
}
sum(5, 3); // a=5, b=3
```

2. Default Parameters (ES6):- Assign default values if no argument or undefined is passed.

```
function greet(name = "Guest") {
  return "Hello " + name;
}
greet();     // Hello Guest
greet("Bob"); // Hello Bob
```

3. Rest Parameters (ES6):- Represent an indefinite number of arguments as an array.

```
function sumAll(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
sumAll(1, 2, 3, 4); // 10
```

4. Arguments Object (Legacy):- Inside every non-arrow function, arguments is an array-like object of all passed arguments.

```
function showArgs() {
  console.log(arguments);
}
showArgs(1, "hello", true);
```

// Output: [1, "hello", true]

Practical Examples

Example 1: Default Parameter

```
function multiply(a, b = 1) {
  return a * b;
}
console.log(multiply(5));    // 5  (b uses default 1)
console.log(multiply(5, 2)); // 10
```

Output:-

5

10

Example 2: Rest Parameter

```
function joinStrings(separator, ...strings) {
  return strings.join(separator);
}
console.log(joinStrings("-", "2024", "05", "22")); // "2024-05-22"
```

Output:-

2024-05-22

Example 3: Using arguments Object

Output:-

Argument 0: apple

Argument 1: banana

```
function listArgs() {
  for (let i = 0; i < arguments.length; i++) {
    console.log(`Argument ${i}: ${arguments[i]}`);
  }
}
listArgs("apple", "banana", "cherry");
```

Summary Table

Parameter Type	Syntax	Purpose	Notes
Regular	(a, b)	Standard parameter passing	Arguments matched by position
Default	(a = 1, b = 2)	Assign default values	Defaults used if argument omitted/undefined
Rest	(...args)	Collect multiple arguments	Gathers extra args into an array
arguments object	Implicit variable in fn	Access all args (array-like)	Not available in arrow functions

50.2 Quiz: Function Parameters

Question 1. What will be the output of the following code?

```
function test(a, b = 3, c = 5) {
  console.log(a + b + c);
}
test(2, undefined, 1);
```

A) 6

B) 8

C) 10

D) NaN

Explanation:

- a = 2 (passed)
- b = undefined → default value 3 used
- c = 1 (passed)
  Sum = 2 + 3 + 1 = 6

Question 2 Which of the following statements about rest parameters is true?

A) Rest parameters allow multiple named arguments.

B) Rest parameters collect extra arguments into an array.

C) Rest parameters are only valid as the first parameter.

D) Rest parameters work like default parameters.

Question 3 What will the following code output?

```
function showArgs() {
  console.log(arguments.length);
}
showArgs(1, 2, 3, 4);
```

A) 0

B) 1

C) 3

D) 4

Question 4 (Code Writing) Write a function called average that:

A) Takes any number of numeric arguments.

B) Returns their average (sum divided by count).

C) Uses rest parameters.

Sample Solution 4:-

```
function average(...nums) {
  if (nums.length === 0) return 0;
  let sum = nums.reduce((acc, n) => acc + n, 0);
  return sum / nums.length;
}
console.log(average(2, 4, 6)); // 4
console.log(average());       // 0
```

51) JavaScript Function Invocation

Function invocation means *calling* or *executing* a function in JavaScript. When you invoke a function, the code inside the function runs.

Why Function Invocation Is Important

- It triggers the function's behavior.
- Controls how this is set inside the function.
- Determines how parameters are passed and processed.

51.1 Four Ways to Invoke Functions in JavaScript

1. Regular Function Invocation

```
function greet() {
  console.log("Hello!");
}
greet(); // Invoked normally
```

This is undefined in strict mode or global object in non-strict mode.

2. Method Invocation

When a function is called as a property of an object.

```
const person = {
  name: "Alice",
  greet: function() {
    console.log("Hi " + this.name);
```

```
  }
};
person.greet();
```

This refers to the object before the dot (person here).

3. Constructor Invocation

Using the new keyword to create an object.

```
function Person(name) {
  this.name = name;
}
const p1 = new Person("Bob");
console.log(p1.name);
```

- Creates a new object.
- this refers to the newly created object.

4. Indirect Invocation (call, apply, bind)

Invoke functions specifying this explicitly.

```
function greet() {
  console.log("Hello " + this.name);
}
const user = { name: "John" };
greet.call(user);  // Hello John
greet.apply(user); // Hello John
const boundGreet = greet.bind(user);
boundGreet();      // Hello John
```

How this Differs in Each Invocation

Invocation Type	How this is set
Regular	Global object or undefined (strict mode)

Invocation Type	How this is set
Method	The object owning the method
Constructor	New object created
Indirect (call/apply/bind)	Explicitly specified object

Invocation Type	Syntax	this Binding	Use Case
Regular	fn()	Global object or undefined (strict mode)	Simple function calls
Method	obj.method()	Object owning the method	Object behavior methods
Constructor	new fn()	New object created	Object creation
Indirect	fn.call(obj), fn.apply(obj), fn.bind(obj)	Explicit object binding	Control over this value

Question 1. What will be the output of this code?

```
function test() {
  console.log(this);
}
test();
```

Solution:

- In strict mode: undefined
- Otherwise: Window or global object

Question 2. What will this refer to in the following?

```
const obj = {
  name: "John",
```

```
  greet() {
    console.log(this.name);
  }
};
const greetFunc = obj.greet;
greetFunc();
```

Solution:
this refers to global object or undefined (strict mode), so output is undefined or error depending on environment.

Question 3. How would you invoke a function fn with this explicitly set to {name: "Alice"}?

Solution:
Use either:

- fn.call({name: "Alice"})
- fn.bind({name: "Alice"})()

Question 4 (Code Writing)

Write a constructor function Person that accepts name and age and sets them as properties. Then create a new Person object and log the name.

Solution:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const p = new Person("Alice", 30);
console.log(p.name); // Alice
```

52) Function Closures

In JavaScript, a **closure** is created when an **inner function** has access to the **outer function's variables**, even after the **outer function has finished execution**. Closures **remember** the scope in which they were created and **preserve data** from the outer function, forming a **lexical environment**. so a closure is:

- A **function** that remembers the **variables from its outer lexical scope**.
- Created **every time a function is created**, at **function creation time**, not execution time.
- Useful when you want to **preserve data** across function calls without exposing it to the global scope.

Closures three levels of scope access:

1. **Local Scope (Own scope):** Variables declared inside the inner function.
2. **Outer Function's Scope:** Variables declared in the outer function.
3. **Global Scope:** Variables declared in the global environment.

Syntax Example

```
function outerFunction() {
  let outerVariable = "I am outside!";
  function innerFunction() {
    console.log(outerVariable);
  }
  return innerFunction;
}
const closureFunc = outerFunction();
closureFunc();  // Output: I am outside!
```

How It Works

- outerFunction() is called and returns innerFunction.
- closureFunc now holds the reference to innerFunction, **which still has access to outerVariable**, even though outerFunction() has completed.

- This is because of **closure** — the innerFunction **remembers the scope** in which it was created.

52.1 Use Cases of Closures

1. Data Encapsulation (Private Variables):

```
function counter() {
  let count = 0;
// count is private and only accessible via the returned function.
  return function() {
    count++;
    return count;
  };
}
const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2
console.log(increment()); // 3
```

2. Function Factories:

```
function greet(name) {
  return function(message) {
    console.log(`${message}, ${name}`);
  };
}
const greetJohn = greet("John");
greetJohn("Hello");  // Output: Hello, John
```

3. SetTimeout Inside Loops (Using Closures):-

```
for (let i = 1; i <= 3; i++) {
  setTimeout(function () {
```

```
    console.log("Count:", i);
// Prints 1, 2, 3 at 1s, 2s, and 3s due to block-scoped let.
  }, i * 1000);
}
```

Example:-

```
<html>
<head>
<title>Nested functions</title>
</head>
<body>
<script>
// closure example
function calculate(x) {
   function multiply(y) {
      return x * y;
   }
   return multiply;
}
const multiply3 = calculate(3);
const multiply4 = calculate(4);
console.log(multiply3); // returns calculate function definition
console.log(multiply3()); // NaN
console.log(multiply3(6)); // 18
console.log(multiply4(2)); // 8
</script></body></html>
```

Explanation:- In the above program, the calculate() function takes a single argument x and returns the function definition of the multiply() function. The multiply() function takes a single argument y and returns x * y.

Both multiply3 and multiply4 are closures.

The calculate() function is called passing a parameter x. When multiply3(6) and multiply4(2) are called, the multipy() function has access to the passed x argument of the outer calculate() function.

52.2 Quiz

Q.1 What will be the output?

```
function testClosure() {
  let x = 5;
  return function() {
    console.log(x);
  };
}
const func = testClosure();
x = 10;
func();
```

Output:- 5

Q. 2 Write a closure that only allows a function to run once.

Solution:-

```
function runOnce(fn) {
  let called = false;
  return function(...args) {
    if (!called) {
      called = true;
      return fn(...args);
    } else {
      console.log("Function already called!");
    }
  };
}
const init = runOnce(() => console.log("Initialized"));
init(); // Initialized
init(); // Function already called!
```

53. Callbacks

In JavaScript, a callback is a function passed as an argument to another function and executed after that function completes. It's a fundamental concept that enables asynchronous programming, allowing operations like reading files, fetching APIs, or user events to occur without blocking the main thread.

Think of callbacks as a way to say: "Do this thing, and once it's done, do this other thing next."

Callback vs Event Handler

An **event handler** is a specialized type of callback that responds to specific events (like clicks or form submissions), while **callbacks** are more general and can be used in any function execution sequence, especially for **asynchronous control flow**.

53.1 Real-Life Analogy

Imagine planning a **birthday party**:

- Tasks: invite guests, order cake, prepare games.
- Final step: hire a **clown** who will **only perform after the guests arrive**.

Here, **"hiring the clown"** is the callback function, and **"guests arriving"** is the task that must complete first. You pass the callback function (clown) to be executed after the main function (guest arrival) completes.

Example 1: Basic Function Calls (No Callbacks Yet)

```
<script>
const funA = () => {
  document.write("Hey from Function A<br>");
}
const funB = () => {
  document.write("Hey from Function B<br>");
}
funA();  //Hey from Function A
funB(); //Hey from Function B   </script>
```

Example 2: Callback Function (Function Passed as Argument)

```
<script>
const funA = (name, callback) => {
  document.write(`Hey ${name}! How are you?<br>`);
  callback(); // Function B as callback
}
const funB = () => {
  document.write("This is Function B executing as a callback.<br>");
}
funA("Preenu", funB);
</script>
```

Output:-

Hey Preenu! How are you?

This is Function B executing as a callback.

Example 3: Callback with Asynchronous Operation (setTimeout)

```
<script>
function fetchData(callback) {
  document.write("Fetching data... Please wait.<br>");
  setTimeout(() => {
    const data = { id: 1, name: "Product A" };
    callback(data); // Callback triggered after 3 seconds
  }, 3000);
}
function processData(data) {
  document.write(`Data received: ID = ${data.id}, Name =
${data.name}<br>`);
}
fetchData(processData);
</script>
```

Output:-

Fetching data... Please wait.

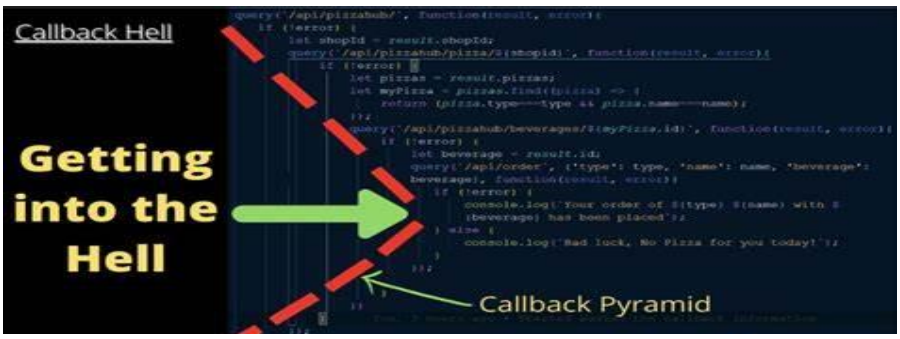
(Data appears after 3 seconds)

Data received: ID = 1, Name = Product A

53.2 What is Callback Hell?

Callback hell (**Pyramid of Doom**) is a situation where multiple asynchronous operations are nested within each other using callbacks. This makes the code:

- Difficult to read
- Harder to debug
- Prone to logical errors



Example:-

```
function getUser(callback) {
  setTimeout(() => {
    callback({ id: 1, name: "John" });
  }, 1000);
}

function getOrders(user, callback) {
  setTimeout(() => {
    callback(["Order1", "Order2"]);
  }, 1000);
}

function getOrderDetails(order, callback) {
  setTimeout(() => {
    callback({ id: order, details: "Details of " + order });
```

```
  }, 1000);
}

// Callback hell begins
getUser(user => {
  getOrders(user, orders => {
    getOrderDetails(orders[0], details => {
      console.log("Order Details:", details);
    });
  });
});
```

Ques:- How to Avoid Callback Hell?

Sol:- You can use:

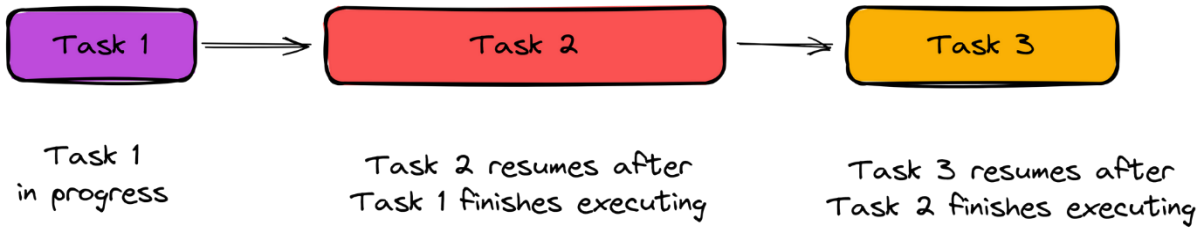
- **Promises**
- **Async/Await**

54. Javascript Asynchronous

Asynchronous programming allows a program to initiate a task and move on to the next one without waiting for the previous task to complete. When the initiated task finishes, it notifies the program (usually through a callback, promise, or event), and the result is handled accordingly.

54.1 Synchronous vs Asynchronous:-

In a synchronous environment, where the request function returns only after it has done its work, the easiest way to perform this task is to make the requests one after the other. This has the drawback that the second request will be started only when the first has finished. The total time taken will be at least the sum of the two response times.



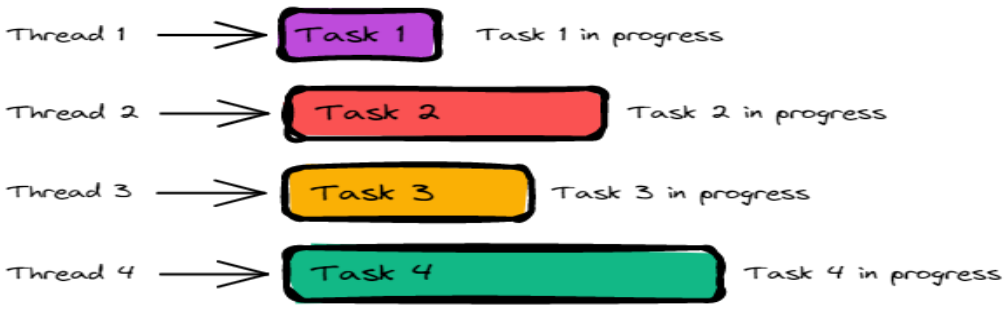
For Example:-let's say that a synchronous program performs a task that requires waiting for a response from a remote server. The program will be stuck waiting for the response and cannot do anything else until the response is returned. This is known as blocking, and it can lead to an application appearing unresponsive or "frozen" to the user

Requirement:

if the program is executed asynchronously, it will continue to run the next line of code instructions rather than becoming blocked. This will enable the program to remain responsive and execute other code instructions while waiting for the timeout to complete.

Key Concepts of Asynchronous:-

- **Non-blocking execution**: JavaScript continues executing code while waiting for time-consuming operations (like fetching data) to complete.
- **Parallel-like behavior** (even in a single-threaded environment): Though JavaScript is single-threaded, asynchronous techniques simulate concurrent task handling.
- **Improved performance & responsiveness**: Programs stay responsive to user inputs and other events while background tasks are in progress.



Real-World Analogy

Think of a chef in a kitchen:

- The chef places a cake in the oven (async task).

- While it bakes, the chef doesn't stand idle — they start chopping vegetables (another task). Once the oven timer rings, the chef takes the cake out.
- This is asynchronous behavior — tasks don't wait in line; instead, they are scheduled and resumed when ready.

Advantages of Asynchronous Programming

1. **Better performance** – Tasks like file reading, HTTP requests, or database queries don't block execution.
2. **Enhanced user experience** – Applications stay smooth and interactive.
3. **Efficient resource utilization** – No CPU cycles are wasted waiting for slow tasks.

Example:-

Here's an example of an asynchronous program using the **setTimeout method:**

```
<script>
console.log("Start of script");

setTimeout(function() {
  console.log("First timeout completed");
}, 2000);
console.log("End of script");
</script>
```

```
Start of script
End of script
First timeout completed
```

Explanation:-

The **setTimeout** method executes a function after a specified time.

The function passed to **setTimeout** will be executed asynchronously, which means that the program will continue to execute the next line of code without waiting for the timeout to complete.

As we can see, **console.log("First timeout completed")** will be executed after 2 seconds. Meanwhile, the script continues to execute the next code statement and doesn't cause any "blocking" or "freezing" behaviour.

55. Javascript Promises

A Promise in JavaScript represents the eventual completion (or failure) of an asynchronous operation and its resulting value. Promises help to avoid deeply nested callbacks and make asynchronous code easier to manage.

A Promise can be in one of the following **three states**.

1. Pending:- initial state
2. Fulfilled state
3. Rejected state

When you request data from a server:

- Initially, the promise is pending.
- Once the data is received successfully, it becomes fulfilled.
- If there's an error (e.g., server down), the promise becomes rejected

Creating a Promise:- You can create a Promise using the **Promise() constructor:**

```
let promise = new Promise(function(resolve, reject) {
  // asynchronous operation
  // call resolve() if successful, or reject() if error
});
```

Example 1: Basic Promise

```
const count = true;
let countValue = new Promise(function(resolve, reject) {
  if (count) {
    resolve("Count value exists.");
  } else {
    reject("No count value found.");
  }
});
console.log(countValue);
```

Output:- Promise {<fulfilled>: "Count value exists."}

Example 2: Promise with Asynchronous Behavior

```
<script>


function func1() {
  return new Promise(function(resolve, reject) {
    setTimeout(() => {
      const error = false;


      if (!error) {
        document.write("Function executed successfully<br>");
        resolve();
      }
else {
        document.write("Function failed<br>");
        reject("Not Worked");
      }
    }, 2000);
  });
}


func1()
  .then(function() {
    document.write("Promise Resolved! Thanks for the help.<br>");
  })


  .catch(function(error) {
    document.write(“ Please try again: " + error + "<br>");
  });


</script>
```

Example 3: Rejected Promise

```
<script>


function func1() {
  return new Promise(function(resolve, reject) {
    setTimeout(() => {
      const error = true;
      if (!error) {
        document.write("Function executed successfully<br>");
        resolve();
      } else {
        document.write("Function failed<br>");
        reject("Not Worked");
      }
    }, 2000);
  });
}


func1()
  .then(function() {
    document.write("Promise Resolved! Thanks for the help.<br>");
  })


 .catch(function(error) {
    document.write("Please try again: " + error + "<br>");
  });


</script>
```

55.1 What is Promise Chaining?

Promise chaining is a technique where you return another promise in a .then() handler. This lets you run asynchronous tasks in sequence.

Example 4: Promise with Callback-like Structure (Chaining)

```
<script>
const students = [
 { name: "Preenu", subject: "JavaScript" },
 { name: "Rohan", subject: "Python" }
];

function enrollStudent(student) {
 return new Promise(function(resolve, reject) {
  setTimeout(function() {
   students.push(student);
   document.write("Student enrolled successfully<br>");
   const error = false;
   if (!error) {
    resolve();
   } else {
    reject("Enrollment Failed");
   }
  }, 1000);
 });
}
function getStudents() {
 setTimeout(function() {
  let str = "";
  students.forEach(function(student) {
   str += `<li>${student.name} (${student.subject})</li>`;
  });
```

```
    document.getElementById('students').innerHTML = str;
    document.write(" 📑 Students fetched successfully<br>");
  }, 2000);
}
const newStudent = { name: "Preeti", subject: "Machine Learning" };
enrollStudent(newStudent)
  .then(getStudents)
  .catch(function(error) {
    document.write("Error: " + error);
  });
</script>
<ul id="students"></ul>
```

55.2 Promise Chaining vs Callback Hell

Callback Hell	Promise Chaining
Deeply nested structure (pyramid)	Flat and readable flow
Difficult to debug and maintain	Easier to read, test, and debug
Poor separation of concerns	Each .then() handles its own logic

Conclusion

- Promises are an elegant replacement for callbacks in async code.
- They help avoid deeply nested structures.
- Promise chaining allows for executing tasks **step by step**, in a readable manner.
- Promises form the foundation for async/await — the most modern way of handling async code.

56. Javascript Async/Await

In JavaScript, the async and await keywords provide a modern and cleaner way to work with asynchronous code, making it behave more like synchronous code.

56.1 Async Keyword

The async keyword is used before a function to indicate that the function always returns a Promise. This makes it easier to work with asynchronous operations.

Syntax:

```
async function functionName(param1, param2, ...) {
  // Statements
}
```

- **functionName:** The name of the asynchronous function.
- **param1, param2**, ...: Parameters passed to the function.

Example 1: Simple Async Function

```
async function f() {
  console.log("Async function.");
  return Promise.resolve(1);
}
f();
```

Output:

Async function.

Explanation:

Even though the function looks like it returns a value, it actually returns a Promise. **You can handle the result using .then():**

```
async function f() {
  console.log('Async function.');
  return Promise.resolve(1);
```

```
}
f().then(function(result) {
  console.log(result);
});
```

Output:

Async function.

1

56.2 await Keyword

The await keyword can only be used inside async functions. It pauses the execution of the async function until the Promise is resolved or rejected.

Syntax:

```
let result = await promise;
```

Example: Using await

```
let promise = new Promise(function (resolve, reject) {
  setTimeout(function () {
    resolve('Promise resolved');
  }, 4000);
});
async function asyncFunc() {
  let result = await promise;
  console.log(result);
  console.log('Hello');
} asyncFunc();
```

Output after 4 seconds:

Promise resolved

Hello

56.3 Working of async/await

- You can use multiple await statements in a single async function.
- Each await waits for its corresponding Promise to resolve before moving on to the next line.

Example: Chained Await

```
let promise1;
let promise2;
let promise3;
async function asyncFunc() {
  let result1 = await promise1;
  let result2 = await promise2;
  let result3 = await promise3;
  console.log(result1);
  console.log(result2);
  console.log(result3);
}
```

Practical Examples

Example 1: Basic Async Function

```
<script>
async function fun() {
  let fun1 = new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve("fun1 promise resolved");
    }, 1000);
  });
  let fun2 = new Promise((resolve, reject) => {
```

```
  setTimeout(() => {
    resolve("fun2 promise resolved");
  }, 5000);
 });
 fun1.then(alert);
 fun2.then(alert);
}
console.log("Welcome to class today");
fun();
</script>
```

Example 2: Using Await Inside Async Function

```
<script>
async function fun() {
 let fun1 = new Promise((resolve) => {
  setTimeout(() => resolve("fun1 promise resolved"), 1000);
 });
 let fun2 = new Promise((resolve) => {
  setTimeout(() => resolve("fun2 promise resolved"), 5000);
 });
 console.log("fun1 is getting loaded...");
 let f1 = await fun1;
 console.log("fun1 has been loaded...");
 console.log("fun2 is getting loaded...");
 let f2 = await fun2;
 console.log("fun2 has been loaded...");
 return [f1, f2];
}
console.log("Welcome to class today");
let a = fun();
```

```
a.then((value) => {
  console.log(value);
});
</script>
```

Example 3: Async with Other Function

```
<script>
async function fun() {
  let fun1 = new Promise((resolve) => {
    setTimeout(() => resolve("fun1 promise resolved"), 1000);
  });
  let fun2 = new Promise((resolve) => {
    setTimeout(() => resolve("fun2 promise resolved"), 5000);
  });
  console.log("fun1 is getting loaded...");
  let f1 = await fun1;
  console.log("fun1 has been loaded...");
  console.log("fun2 is getting loaded...");
  let f2 = await fun2;
  console.log("fun2 has been loaded...");
  return [f1, f2];
}
const myfun2 = () => {
  console.log("I'm another function");
};
console.log("Welcome to class today");
let a = fun();
let b = myfun2();
a.then((value) => {
  console.log(value);
```

```
});
</script>
```

Example 4: All Async Functions

```
<script>
async function fun() {
  let fun1 = new Promise((resolve) => {
    setTimeout(() => resolve("fun1 promise resolved"), 1000);
  });
  let fun2 = new Promise((resolve) => {
    setTimeout(() => resolve("fun2 promise resolved"), 5000);
  });
  console.log("fun1 is getting loaded...");
  let f1 = await fun1;
  console.log("fun1 has been loaded...");
  console.log("fun2 is getting loaded...");
  let f2 = await fun2;
  console.log("fun2 has been loaded...");
  return [f1, f2];
}
const myfun2 = async () => {
  console.log("I'm another async function");
};
const outer = async () => {
  console.log("Welcome to class today");
  let a = await fun();
  let b = await myfun2();
};
outer();
</script>
```

Summary

Keyword	Purpose
async	Declares a function as asynchronous and always returns a Promise
await	Pauses execution until the Promise resolves

57. JavaScript DOM Methods

The DOM (Document Object Model) is a programming interface for HTML and XML documents. It represents the structure of a document as a tree of nodes, where each node is an object representing a part of the page. JavaScript uses the DOM to access and manipulate HTML documents dynamically.

Why DOM Methods Are Used

DOM methods allow developers to interact with and modify the structure, style, and content of web pages. These methods are essential for dynamic client-side scripting, such as form validation, interactive UI, and real-time content updates.

57.1 Common DOM Methods

Method	Description
getElementById()	Returns the element with the specified ID.
getElementsByClassName()	Returns all elements with a specified class name.
getElementsByTagName()	Returns all elements with the specified tag name.
querySelector()	Returns the first element that matches a CSS selector.
querySelectorAll()	Returns all elements that match a CSS selector.
createElement()	Creates a new HTML element.
appendChild()	Adds a new child node to an element.
removeChild()	Removes a specified child node.

Method	Description
replaceChild()	Replaces a child node with a new node.
setAttribute()	Sets a new value to an attribute.
getAttribute()	Returns the value of an attribute.
innerHTML	Gets or sets the HTML content of an element.
textContent	Gets or sets the text content of an element.

Example 1: Accessing an Element by ID

```
<!DOCTYPE html>
<html>
<body>
 <h1 id="title">Welcome</h1>
 <script>
  let heading = document.getElementById("title");
  console.log(heading.textContent); // Output: Welcome
 </script>
</body>
</html>
```

Example 2: Changing Content Using innerHTML

```
<!DOCTYPE html>
<html>
<body>
 <p id="demo">Hello!</p>
 <script>
  document.getElementById("demo").innerHTML = "Goodbye!";
 </script>
</body>
</html>
```

Example 3: Creating and Appending an Element

```
<!DOCTYPE html>
<html>
<body>
```

```
  <ul id="list"></ul>
  <script>
    let li = document.createElement("li");
    li.textContent = "Item 1";
    document.getElementById("list").appendChild(li);
  </script>
</body>
</html>
```

Use Cases

- Adding/removing items in a list dynamically
- Validating user input and displaying messages
- Creating interactive elements like modals, tooltips
- Modifying page content based on user actions or API results

58. JavaScript DOM Elements

DOM elements are individual HTML elements (like , <p>, <ul>, <input>, etc.) represented as objects in the DOM tree. These objects provide properties and methods that allow JavaScript to interact with and manipulate them.

Each element in an HTML document becomes a DOM element node, and JavaScript can access and modify these elements to create dynamic web pages.

Why DOM Elements Are Important

DOM elements are the foundation of any webpage. Accessing and modifying them allows developers to:

- Change content
- Update styles
- Handle user interactions
- Add or remove components dynamically

58.1 Types of DOM Elements

Element Type	Example
HTML Element	, , <p>
Form Element	<input>, <textarea>, <select>
Media Element	<img>, <audio>, <video>
Table Element	<table>, <tr>, <td>
Document Root	<html>, <body>, <head>

58.2 Useful Properties of DOM Elements

Property	Description
innerHTML	Gets or sets HTML content of an element
textContent	Gets or sets plain text of an element
className	Gets or sets the value of the class attribute
style	Used to set inline CSS styles
value	Used to get/set input field values

59. JavaScript DOM Events

DOM Events are actions or occurrences that happen in the browser, which JavaScript can detect and respond to. These events are usually triggered by user interactions like clicks, key presses, mouse movements, form submissions, or even page loading.

Why DOM Events Are Important

- Make web pages interactive
- Respond to user inputs
- Validate form entries
- Create animations or dynamic UI changes
- Monitor and react to browser behavior

59.1 Common Types of DOM Events

Event Type	Description
click	When an element is clicked
mouseover	When the mouse pointer is over an element
mouseout	When the mouse pointer leaves an element
keydown	When a key is pressed
keyup	When a key is released
load	When the page or an image finishes loading
submit	When a form is submitted
change	When input, select, or textarea value changes
focus	When an element gets focus
blur	When an element loses focus

59.2 How to Attach Events

1. Inline Event Handling

```
<button onclick="alert('Button clicked')">Click Me</button>
```

2. JavaScript Property Method

```
<button id="btn">Click Me</button>
<script>
  document.getElementById("btn").onclick = function() {
    alert("Clicked using JavaScript");
  };
</script>
```

3. Using addEventListener() (Recommended)

```
<button id="btn">Click Me</button>
<script>
```

```
  const button = document.getElementById("btn");
  button.addEventListener("click", function() {
    alert("Clicked using addEventListener");
  });
</script>
```

Example 1: Click Event

```
<!DOCTYPE html>
<html>
<body>
  <p id="text">Click the button to change this text.</p>
  <button onclick="changeText()">Click Me</button>
  <script>
    function changeText() {
      document.getElementById("text").textContent = "Text changed!";
    }
  </script>
</body>
</html>
```

Example 2: Mouseover and Mouseout

```
<!DOCTYPE html>
<html>
<body>
  <div id="box" style="width:100px;height:100px;background:red;"></div>
  <script>
    const box = document.getElementById("box");
    box.onmouseover = function() {
      box.style.backgroundColor = "green";
    };
    box.onmouseout = function() {
      box.style.backgroundColor = "red";
    };
  </script>
</body>
</html>
```

Example 3: Keydown Event

```
<!DOCTYPE html>
<html>
<body>
 <input type="text" onkeydown="alert('Key pressed!')">
</body>
</html>
```

Example 4: Form Submit Event

```
<!DOCTYPE html>
<html>
<body>
 <form onsubmit="return validateForm()">
  <input type="text" id="name" required>
  <input type="submit" value="Submit">
 </form>
 <script>
  function validateForm() {
   const name = document.getElementById("name").value;
   if (name === "") {
    alert("Name is required!");
    return false;
   }
   alert("Form submitted");
   return true;
  }
 </script>
</body></html>
```

60. JavaScript DOM Event Listener

An event listener is a function in JavaScript that waits (or "listens") for a specific event to occur on an HTML element and then executes a specified action when that event occurs.

The most commonly used method to register an event listener is:

```
element.addEventListener(event, function, useCapture);
```

Syntax:-

```
element.addEventListener("event", functionName, useCapture);
```

Parameter	Description
event	Required. A string that specifies the name of the event, like "click"
functionName	Required. The function to run when the event occurs
useCapture	Optional. Boolean indicating event propagation phase (default is false)

Why Use addEventListener()?

- It allows multiple event handlers on the same event and element.
- It supports separation of HTML and JavaScript (cleaner code).
- It works well with removing listeners via **removeEventListener.**

Example 1: Basic Click Event Listener

```
<!DOCTYPE html>
<html>
<body>
 <button id="btn">Click Me</button>
 <script>
  document.getElementById("btn").addEventListener("click", function() {
   alert("Button was clicked!");
  });
 </script>
</body></html>
```

Example 2: Multiple Event Listeners on Same Element

```
<!DOCTYPE html>
<html>
<body>
 <button id="multiBtn">Hover or Click Me</button>
 <script>
  const btn = document.getElementById("multiBtn");
  btn.addEventListener("click", function() {
   console.log("Clicked!");
```

```
  });
  btn.addEventListener("mouseover", function() {
    console.log("Mouse over!");
  });
 </script>
</body>
</html>
```

Example 3: Passing a Named Function

```
<!DOCTYPE html>
<html>
<body>
 <button id="logBtn">Log Message</button>
 <script>
  function logMessage() {
    console.log("Event triggered!");
  }

  document.getElementById("logBtn").addEventListener("click",
logMessage);
 </script>
</body>
</html>
```

Example 4: Removing an Event Listener

```
<!DOCTYPE html>
<html>
<body>
 <script>
  function sayHi() {
    alert("Hello!");
    document.getElementById("clickBtn").removeEventListener("click",
sayHi);
  }
  document.getElementById("clickBtn").addEventListener("click", sayHi);
 </script>
</body>
</html>
```

In this example, the alert shows only once, then the event listener is removed.

Example 5: Event Listener with this

```
<!DOCTYPE html>
<html>
<body>
 <button id="colorBtn">Change Color</button>
 <script>
  document.getElementById("colorBtn").addEventListener("click",
function() {
    this.style.backgroundColor = "orange";
  });
 </script>
</body>
</html>
```

Conclusion

Using addEventListener() is the best practice for handling DOM events because it's flexible, supports multiple listeners, and keeps your JavaScript separate from your HTML structure. It's essential for building responsive and interactive web applications.

61. JavaScript DOM Nodes

In the DOM **(Document Object Model),** everything is a node. A node is an object that represents part of a web page.

61.1 Types of DOM Nodes

Node Type	Description	Example
Document Node	Represents the entire document	document
Element Node	Represents an HTML element	<p>, , , etc.
Text Node	Represents the text inside an element	"Hello World" inside <p>
Attribute Node	Represents an attribute of an element	class="main"
Comment Node	Represents a comment in HTML	<!-- Comment -->

61.2 DOM Node Tree

The DOM represents a page as a tree structure of nodes. Example:

```
<!DOCTYPE html>
<html>
 <body>
  <div>Hello <span>World</span></div>
 </body>
</html>
```

61.3 Node Tree Representation:

- Document
    - html
        - body
            - div (Element Node)
                - Text Node: "Hello"
                - span (Element Node)
                    - Text Node: "World"

61.4 Common Node Properties

Property	Description
nodeName	Returns the name of the node
nodeType	Returns the type of node (1 = Element, 3 = Text)
nodeValue	Returns the value (used mostly for Text Nodes)
childNodes	Returns a NodeList of child nodes
firstChild	Returns the first child node
lastChild	Returns the last child node
parentNode	Returns the parent of the node

Property	Description
nextSibling	Returns the next node at the same level
previousSibling	Returns the previous node at the same level

Example 1: Accessing Node Information

```
<!DOCTYPE html>
<html>
<body>
 <p id="demo">Hello World</p>
 <script>
  const node = document.getElementById("demo");
  console.log(node.nodeName);   // P
  console.log(node.nodeType);   // 1
  console.log(node.firstChild.nodeValue);  // Hello World
 </script>
</body>
</html>
```

Example 2: childNodes and Traversing

```
<!DOCTYPE html>
<html>
<body>
 <ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
 </ul>

 <script>
  const list = document.getElementById("list");
  const nodes = list.childNodes;
  nodes.forEach(node => {
   console.log("Node Type:", node.nodeType, "Node Name:",
node.nodeName);
  });
 </script>
</body>
</html>
```

Note: childNodes includes **text nodes** (like whitespaces), not just elements.

Example 3: Differentiating Element and Text Nodes

```
<!DOCTYPE html>
<html>
<body>
 <div id="box">
   Hello <span>World</span>
 </div>
 <script>
   const box = document.getElementById("box");
   console.log(box.childNodes); // [Text, span, Text]
   console.log(box.children);   // [span]
 </script>
</body>
</html>
```

- childNodes: includes all nodes (text, comment, element)
- children: includes only **element nodes**

Node Types Table

Node Type Code	Node Type Name
1	Element
2	Attribute
3	Text
8	Comment
9	Document

Conclusion

DOM Nodes are the basic building blocks of the DOM tree. Understanding how to work with nodes allows developers to traverse and manipulate HTML documents effectively. Nodes include elements, text, comments, and more — and they enable powerful document structure control using JavaScript.